

ABSTRACT

Ensuring personal safety, especially for women, remains a major concern in today's society. This project proposes a smart safety device built using Internet of Things (IoT) technology that can quickly respond during emergency situations. The system combines different sensors and modules to detect danger and communicate alerts effectively. It uses an MPU6050 sensor to monitor movement and identify falls, a GPS module (NEO-6M) to track the user's location, and a SIM800L module to send messages and make calls. An ESP32-CAM is used to capture images, while a XIAO ESP32-S3 with a microphone enables voice-based activation. All these components are controlled by an ESP32 microcontroller.

The device is designed with multiple ways to trigger an alert so that it can work reliably in different situations. The user can press a panic button, use a voice command like "HELP," or rely on automatic fall detection based on sudden movement or tilt. Once triggered, the system collects location data and captures images of the surroundings to provide more information about the situation.

Alert messages are then sent to selected contacts through SMS and phone calls. If Wi-Fi is available, the captured images are also shared through Telegram. To avoid unnecessary panic, the system includes an option to cancel the alert or send a message confirming that the user is safe. Overall, this device aims to offer a practical and dependable safety solution by combining multiple technologies to ensure faster and more effective emergency response.

Keywords: Internet of Things (IoT), personal safety device, women's security, MPU6050, SIM800L, NEO-6M GPS, ESP32-CAM, voice activation, fall detection, emergency alert systems, machine learning, GSM communication, Telegram.

CONTENT

<i>Abstract</i>	<i>ii</i>
<i>Contents</i>	<i>iv</i>
<i>List of Figures</i>	<i>v</i>
CHAPTER 1 - INTRODUCTION	1-7
1.1 Overview	1-3
1.2 Problem Definition	3-4
1.3 Scope of the Project	4-5
1.4 Aim and Objectives	5-6
1.5 Thesis Outline	6-7
 CHAPTER 2 - LITERATURE REVIEW	 8-13
2.1 Introduction	8-11
2.2 Research Gap	11-13
 CHAPTER 3 - PROPOSED METHODOLOGY	 14-20
3.1 Introduction	14-15
3.2 Block Diagram	15-17
3.3 Flowchart	18-20
 CHAPTER 4 - HARDWARE AND SOFTWARE TOOLS	 21-36
4.1 Hardware Tools Used	21-33
4.2 Software Tools Used	33-36
CHAPTER 5 - RESULTS AND DISCUSSIONS	37-44
5.1 GSM & GPS-Based Emergency Alert System	37-38
5.2 Response Time and Latency	38-39
5.3 GPS Precision	39-40
5.4 Image Capture and Transmission Performance	40-41
5.5 False Alarm Mitigation	41-43
5.6 Applications	44
CHAPTER 6 - CONCLUSION AND FUTURE SCOPE	45-46
6.1 Conclusion	45-46
6.2 Future Scope	46
REFERENCES	47-49
 APPENDIX – A	 52-58
GSM & GPS-Based Emergency Alert System	
APPENDIX B	59-69
Voice Recognition Module	
APPENDIX C	70-83
Wi-Fi & Camera-Based Alert System	

LIST OF FIGURES

Figure no.	Description	Page no.
3.1	Proposed system block diagram	15
3.2	Circuit Diagram	17
3.3	Flowchart	18
4.1	ESP32 Microcontroller	21
4.2	XIAO ESP32 S3	22
4.3	SIM 800L V2.0 GSM	23
4.4	GPS MODULE	24
4.5	ESP32 CAM	25
4.6	MPU6050	26
4.7	Power bank	27
4.8	Rocket Switch	28
4.9	Slide Switch	29
4.10	LED	30
4.11	Battery	31
4.12	Push button	32
4.13	Arduino IDE	33
5.1	Result	37
5.2	Call Alert	39
5.3	GPS Location	40
5.4	Captured Image	41
5.5	Safe Messages	42

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

The rapid growth of embedded systems and Internet of Things (IoT) technologies has significantly transformed the way safety and monitoring solutions are designed and implemented. With increasing urbanization and changing social dynamics, ensuring personal safety—particularly for women—has become a critical concern. Traditional safety measures often rely on manual intervention or external assistance, which may not always be available during emergency situations. This has led to a growing need for intelligent, autonomous systems capable of detecting threats and initiating timely responses.

Modern safety systems are expected to operate across multiple domains, combining sensing, processing, and communication capabilities into a compact and efficient platform. Such systems must be capable of interpreting real-world conditions, identifying potential risks, and transmitting alerts through reliable communication channels. The integration of heterogeneous components—including sensors, communication modules, and processing units—introduces challenges related to coordination, timing, and data management. Addressing these challenges requires a system-level design approach that ensures seamless interaction between different hardware and software elements.

IoT-based safety devices have emerged as a promising solution to these challenges. By leveraging real-time data acquisition and wireless communication technologies, these systems enable continuous monitoring and rapid response in critical situations. Sensors such as accelerometers and gyroscopes can be used to detect abnormal motion patterns, including falls or sudden impacts, while positioning systems provide accurate location information. Communication modules, particularly those based on cellular networks, allow alerts to be transmitted even in the absence of local internet connectivity, thereby improving system reliability in diverse environments.

Despite these advancements, designing a dependable safety device involves several practical considerations. Emergency situations may arise in unpredictable conditions, where a single mode of activation or communication may not be sufficient. Systems that rely solely on manual triggering can fail if the user is unable to respond, while those based only on automatic detection may generate false alarms. Therefore, it becomes essential to incorporate multiple triggering mechanisms and communication pathways to enhance system robustness and usability.

Another important aspect is the ability to provide contextual information during an emergency. Location data alone may not always be sufficient to assess a situation accurately. The inclusion of additional inputs, such as visual evidence or audio-based triggers, can significantly improve the effectiveness of the response. At the same time, mechanisms must be in place to minimize false alerts and allow users to retain control over the system when necessary.

The development of such systems benefits from a modular and application-driven design methodology. In this approach, system functionality is divided into well-defined components, such as sensing, decision-making, and communication units. This modular structure simplifies system integration, testing, and future enhancements. It also enables designers to implement customized logic for data processing, event detection, and alert generation based on specific application requirements.

The present work follows this design philosophy and focuses on the development of an IoT-enabled personal safety device for women. The system integrates multiple sensors and communication modules to support real-time monitoring and emergency response. Key features include manual activation through a panic button, voice-based triggering using keyword detection, and automatic activation through motion analysis. Upon detecting an emergency, the device gathers location data and captures environmental information, which is then transmitted to predefined contacts using both cellular and internet-based communication methods.

Rather than relying on a single point of interaction, the proposed system emphasizes redundancy and adaptability in its design. By combining multiple sensing techniques and communication channels, it aims to ensure consistent performance under varying

conditions. The modular architecture adopted in this work also facilitates systematic testing and validation of individual components, thereby improving overall system reliability.

1.2 PROBLEM DEFINITION

In recent years, incidents related to personal safety, especially involving women, have highlighted the limitations of conventional security measures. Existing safety solutions often depend on manual reporting or external assistance, which may not be immediately accessible during critical situations. This delay in response can reduce the effectiveness of emergency support systems and increase vulnerability in real-world scenarios.

Another major limitation of traditional approaches is the lack of real-time monitoring and intelligent decision-making capabilities. Many available systems are either fully manual, requiring user intervention, or rely on a single detection mechanism, which may not be sufficient in diverse emergency conditions. For example, a system based only on manual activation may fail if the user is unable to physically trigger an alert, while systems relying solely on automatic detection may generate false alarms due to environmental or accidental movements.

Additionally, existing solutions often operate using a single communication channel, which can lead to failure in areas with poor network coverage or connectivity issues. The absence of redundancy in communication and alert mechanisms reduces system reliability during emergencies. Furthermore, most conventional systems do not provide contextual information such as real-time location updates or visual evidence, which are crucial for faster and more accurate response from emergency contacts or authorities.

Therefore, there is a clear need for a compact, intelligent, and reliable safety system that integrates multiple sensing techniques, supports both manual and automatic activation, and ensures effective communication through different channels. Such a system should be capable of detecting emergencies in real time, minimizing false alarms, and providing accurate situational data to improve response efficiency.

The problem addressed in this work is the design and development of an IoT-enabled personal safety device that overcomes the limitations of existing systems by combining

multi-modal triggering mechanisms, real-time location tracking, and multi-channel communication to ensure timely and reliable emergency response.

1.3 SCOPE OF THE PROJECT

The main aim of this project is to create a small, personal safety device that uses IoT technology. It combines several important features like sensing, communication, and an alert system into one device. The device can detect emergencies in different ways, such as manually, through sound, or by motion. It uses GPS to track the location of the object, sends alerts through GSM communication, and also has a camera.

The system can work both when connected to the internet and when it isn't, so it functions well even in areas without a network. The device is designed to be small and easy to carry for regular use. Before it's used in real life, the prototype will be tested in controlled settings to make sure it works properly.

Once it's deployed in real situations and scaled up, it can be improved further to respond better in emergencies.

Scope of the project includes:

- Development of a multi-mode emergency triggering system using manual panic button activation, voice-based keyword detection, and automatic fall or motion detection.
- Integration of an MPU6050 sensor for motion analysis to identify sudden movements or fall conditions.
- Implementation of GPS-based location tracking using the NEO-6M module to provide real-time position updates during emergencies.
- Use of GSM communication (SIM800L module) to send SMS alerts and initiate voice calls to predefined contacts.
- Integration of ESP32-CAM to capture and transmit images for providing visual context during emergency situations.
- Implementation of voice recognition capability using a microphone-based input system for detecting emergency keywords such as “HELP.”

- Development of a centralized control system using ESP32 to coordinate all sensors, communication modules, and processing tasks.
- Provision for sending alerts through internet-based platforms (such as Telegram) when Wi-Fi connectivity is available.
- Design of a user-friendly and portable system suitable for real-time personal safety applications.
- Ensuring modular architecture to support future upgrades and additional feature integration

1.4 AIM AND OBJECTIVES

AIM:

The aim of this project is to develop an IoT-based personal safety device for women that can identify emergency situations in real time and immediately communicate alerts to trusted contacts. The system is designed to improve personal security by combining sensing, processing, and communication functions in a single embedded solution. It focuses on providing quick assistance during critical moments by supporting both automatic and manual alert activation. The project also aims to ensure that essential information such as the user's location and surrounding conditions can be shared efficiently, enabling faster response and better situational awareness during emergencies.

OBJECTIVES:

- To develop a compact and portable safety device using IoT technology for real-time emergency response.
- To implement multiple emergency triggering methods such as panic button activation, voice-based detection, and automatic fall detection.
- To integrate motion sensing using the MPU6050 sensor for identifying abnormal movements or falls.
- To enable real-time location tracking using the NEO-6M GPS module.
- To design a reliable communication system using GSM technology for sending SMS alerts and making emergency calls.

- To capture and transmit images using ESP32-CAM for providing situational awareness during emergencies.
- To incorporate voice recognition for detecting emergency keywords such as “HELP.”
- To develop a centralized control system using ESP32 for coordinating all modules efficiently.
- To ensure fast, reliable, and user-friendly emergency alert generation for improved personal safety.

1.5 THESIS OUTLINE

The following outline provides a structure for the thesis work, broken down into six chapters that describe the development of the project from the very first idea to the final assessment.

Chapter 1: Introduction

This first and foremost chapter offers a background to the topic, pointing out the urgent necessity in the development of better safety solutions for personal usage in today's world especially focusing on the security of women. Besides identifying the main issue being addressed by this project; it also lists the specific goals, objectives, and limitations. The chapter outlines how the proposed IoT-based solution will be implemented and the rationale behind its multi-modal feature.

Chapter 2: Literature Survey

This part discusses the present status of personal safety related as technology, and stops at the standard solutions available today such as mobile applications which can be a very dedicated GSM wearables. This chapter stretches from the study of past works to the analysis of the existing products, and by identifying the shortcomings of present solutions, e.g. lack of reliability or limited trigger methods, it sets up the research gaps this project intends to cover.

Chapter 3: Proposed Methodology

The third chapter is about the design and functional logic of the system. It describes how different units such as GPS, GSM, camera, and voice recognition are brought together to form a single integrated entity. Besides, I discuss the principles behind the use of different trigger methods, that is, the manner in which the system moves from idle to the state of alarm when an emergency is detected.

Chapter 4: Hardware and Software Implementation

The chapter describes the hardware setup of the project along with the electronic circuit design. Each sensor and module interfacing method has been discussed in detail. The major points remain embedded programming and communication protocols on the software front which through the voice recognition model and emergency alert sequences provide the desired functionalities.

Chapter 5: Results and Discussion

The chapter presents the collected data during the testing and evaluation phases. This section provides an in-depth analysis of the system's performance, in particular, measuring its response time, accuracy of the triggers, and its reliability overall. Then the results are discussed with a view towards existing technologies highlighting the advantages of this multi-modal method in terms of practicality.

Chapter 6: Conclusion and Future Scope

The concluding chapter recaps the major discoveries of the project and considers how effective the design was in personal safety incidents in the real world. It also proposes hopeful areas for future research, e.g. the employment of more intricate threat detection AI, designing a smaller size for better need, and system integration with mobile applications that are more advanced.

CHAPTER 2

LITERATURE REVIEW

2.1 INTRODUCTION

[1] The IEEE paper, “System for Women’s Security using IoT” was published by T. Sunitha, B. Rama Devi, A. Swathi, D. Ashok, and Dr. B. Mouli Chandra in 2020. This study introduces an IoT-based device aimed at enhancing women’s safety. The system is built around a Raspberry Pi controller and integrates multiple sensors, such as heart rate, temperature, GPS, a camera, and a panic button for manual activation. When abnormal readings are detected or the panic button is pressed, real-time alerts including location data and visual evidence are sent to predefined contacts. The inclusion of the camera allows for capturing critical evidence, and GPS ensures accurate tracking. This foundational work guides the development of multi-modal triggering mechanisms beyond simple manual activation (T. Sunitha et al., 2020).

[2] The IEEE paper, “Wearable Fall Detection System with Real-Time Localization and Notification Capabilities” was published by Chin-Kun Tseng, Shi-Jia Huang, Lih-Jen Kau, and co-authors in 2025. This research presents a wearable device capable of autonomously detecting falls using accelerometer and gyroscope sensors. Machine learning algorithms distinguish between routine movements and actual falls, triggering real-time alerts with GPS coordinates to emergency contacts. Despite its accuracy in fall detection, the device lacks voice-activated triggers, false alarm cancellation, and hybrid communication methods, limiting its effectiveness in diverse emergency scenarios (C.-K. Tseng et al., 2025).

[3] The IEEE paper, “IoT Based Automatic Women’s Safety Device for Enhanced Personal Security” was published by Arati Nilesh Kane, Tabassum Maktum, Vanita Mane, Namita Pulgam, and co-authors in 2024. This system employs a Raspberry Pi and integrates heart rate and temperature sensors for automatic emergency detection. Alerts are sent through network connectivity. Although useful for health-related emergencies, it does not support

multiple trigger modes, camera-based evidence, or hybrid Wi-Fi/GSM communication, which limits overall reliability (A. N. Kane et al., 2024).

[4] The IEEE paper, “IoT-Based Voice Assistant using Raspberry Pi and Natural Language Processing” was published by Rajakumar P., K. Suresh, M. Boobalan, Archana R., and co-authors in 2023. The study demonstrates offline voice recognition on Raspberry Pi devices, enabling reliable voice command execution without internet dependency. However, it does not include fall detection, physiological monitoring, or automatic alerting, restricting its applicability in comprehensive safety systems (R. P. Rajakumar et al., 2023).

[5] The IEEE paper, “A Fall Detection Device Based on Single Sensor Combined with Joint Features” was published by Li Zhang, Yu-An Liu, Qiuyu Wang, Huilin Chen, Jingao Xu, and co-authors in 2023. The system uses a single accelerometer with joint-feature extraction and machine learning to improve fall detection accuracy. While detection is effective, it lacks alert mechanisms such as voice triggers, SMS/GSM backups, or image capture, which are essential for timely emergency response (L. Zhang et al., 2023).

[6] The IEEE paper, “Design and Implementation of an Android App for Real-Time Location Monitoring and Emergency Assistance” was published by Rama Chaithanya Tanguturi, Dankan Gowda V., Sk. Heena Kauser, N. Vyshnavi, K. Hephzibah, and T. Aswani in 2023. This research proposes a smartphone application for monitoring user location and sending emergency alerts. While effective for manual reporting, it depends entirely on smartphone availability and lacks autonomous sensor-based detection, limiting usability when the user is incapacitated (R. C. Tanguturi et al., 2023).

[7] The IEEE paper, “Smart Security Device for Women Based on IoT Using Raspberry Pi” was published by Tejonidhi M. R., Aishwarya, Chaithra K. S., Dayana M. K., and Nagamma H. in 2019. The system integrates a buzzer, camera, flex sensor, and GPS/GSM modules to detect emergencies and send alerts. While it effectively captures visual evidence and communicates alerts, the device primarily relies on manual activation and does not include automated or multi-modal triggering mechanisms, which can delay response in critical situations (T. M. R. Tejonidhi et al., 2019).

[8] The IEEE paper, “ProTecht – Implementation of an IoT Based 3-Way Women Safety Device” was published by Trisha Sen, Arpita Dutta, S. Singh, and Vaegae Nveen Kumar in 2019. This system employs several triggers, including a sound sensor, alongside Raspberry Pi and GSM modules for sending alerts. Although it improves responsiveness compared to single-trigger devices, its ability to capture visual evidence is limited, and the integration of advanced sensors is minimal, which restricts its overall effectiveness during emergencies (T. Sen et al., 2019).

[9] The IEEE paper, “AI-Driven Threat Response using Voice Recognition and IoT” was published by Irene Lu and Richard Guo in 2025. This study focuses on using offline voice recognition integrated with IoT devices to trigger alerts through vocal commands. While this approach improves response during panic situations, the system does not include fall detection capabilities or combined communication methods, limiting its applicability for comprehensive personal safety solutions (I. Lu & R. Guo, 2025).

[10] The IEEE paper, “Fall Detection Systems for Internet of Medical Things” was published by Anbarasu and co-authors in 2025. The researchers developed a fall detection mechanism using accelerometer and gyroscope sensors within an IoT framework. Although the device accurately identifies falls, the absence of GSM or Wi-Fi modules reduces its effectiveness in practical emergency settings (Anbarasu et al., 2025).

[11] The IJISRT paper, “Real-Time Human Fall Detection and Alert System” was published by the IJISRT Research Team in 2025. This system employs sensors to detect falls and instantly sends alerts to designated contacts. However, it lacks both voice recognition and visual evidence recording features, which restricts its utility beyond medical monitoring scenarios (IJISRT Research Team, 2025).

[12] The IEEE paper, “Smart Security Device for Women Based on IoT Using Raspberry Pi” was published by Tejonidhi M. R., Aishwarya, Chaithra K. S., and co-authors in 2021. The device combines GPS, GSM, and camera modules to provide location tracking and visual documentation. Nonetheless, it relies on manual activation and does not feature automated anomaly detection or intelligent alert triggering, which can reduce efficiency in urgent situations (T. M. R. Tejonidhi et al., 2021).

[13] The ResearchGate paper, “IoT-Based Personal Safety Devices: Key Features and Functionalities” was published by Falade Adeola in 2023. This review evaluates existing IoT safety devices, highlighting shortcomings in multi-modal activation, combined communication methods, and the collection of visual evidence. The study emphasizes the importance of developing more reliable and advanced safety systems (F. Adeola, 2023).

[14] The IEEE paper, “Raspberry Pi Based Women Safety System” was published by Nalina H. D., Aishwarya B., Harshitha P., Kruthika M., and P. Rachana Naidu in 2023. The authors designed a system that uses fingerprint authentication along with GSM alerts to inform nearby contacts. While effective for rapid alerting, the device does not support alternative triggers such as voice commands or fall detection, nor does it include camera-based evidence capture, limiting its overall effectiveness (N. H. D. Nalina et al., 2023).

[15] The IEEE paper, “A Real-Time IoT Based Fall Detection and Alert System for Elderly” was published by Shruti Ghosh, Rounak Ghosal, Ahana Saha, Niladri Mukherjee, and Prof. Sudip Dogra in 2025. This Raspberry Pi-based system monitors falls using accelerometer and gyroscope sensors, sending alerts with GPS coordinates to caregivers. Despite its accuracy, the device lacks hybrid Wi-Fi and GSM communication and voice-based alerting, which can limit its responsiveness in diverse emergencies (S. Ghosh et al., 2025).

[16] The IEEE paper, “IoT-Based Wearable Devices for Personal Safety and Accident Prevention” was published by Oleg Yulia and Denis Sofia in 2025. The wearable IoT system monitors the surrounding environment using multiple sensors and Raspberry Pi to prevent accidents and provide immediate alerts. Although effective in alerting and monitoring, it does not include camera or audio evidence collection and lacks multi-modal trigger options, which are critical for full personal safety coverage (O. Yulia & D. Sofia, 2025).

2.2 RESEARCH GAP

After reviewing numerous studies on IoT-enabled personal safety devices, it is clear that substantial progress has been made in areas such as wearable safety technologies, fall detection systems, voice-triggered alerts, and GPS-based monitoring [1–16]. Many of these

devices successfully integrate sensors, communication modules, and real-time monitoring features to enhance user safety. For example, Raspberry Pi- and ESP32-based platforms have demonstrated effective alert transmission through GSM or Wi-Fi networks, while fall detection using accelerometer and gyroscope sensors has improved emergency responsiveness. Additionally, voice-activated systems enhance usability during panic situations, and camera modules provide critical visual evidence that supports verification and intervention.

Despite these advancements, several limitations remain in the current approaches:

Limited Multi-Modal Triggering: Many devices depend on a single trigger type, such as a manual panic button or fall detection sensor [12, 14]. This reliance can reduce effectiveness when the user is unable to activate the device manually or when a single sensor may generate false positives or negatives.

Insufficient Integration of Hybrid Communication: While certain systems employ either GSM or Wi-Fi [13, 15], few combine both to ensure consistent alert delivery under varying network conditions. This can lead to delayed notifications during critical emergencies.

Inadequate Evidence Capture: Although some devices include camera modules [11, 12], many fail to pair real-time image capture with automatic triggering. Without visual verification, emergency responders may struggle to accurately assess the situation, potentially delaying timely intervention.

False Alarm Management: Most existing systems lack mechanisms to cancel false alarms or confirm the user's safety after accidental activation [12, 14]. This can cause unnecessary panic among contacts or responders, reducing overall system reliability.

Complexity and Usability Challenges: Several solutions are technically complex and require expertise for proper use and maintenance [13, 16]. In emergencies, simplicity is essential, and devices that are difficult to operate may not achieve their intended purpose.

Limited Modularity and Upgradability: Many devices are not designed for modular integration, making it difficult to add new sensors or communication modules [14, 15].

This limits the ability to incorporate future enhancements, such as machine learning-based anomaly detection or improved location tracking.

Absence of Comprehensive Safety Systems: Most research focuses on isolated aspects of personal safety, such as fall detection [15], voice activation [9], or GPS monitoring [12], without creating a unified, multi-modal solution. A holistic approach is needed to handle diverse emergency scenarios effectively.

Considering these gaps, there is a clear demand for a comprehensive IoT-based personal safety device that combines multiple triggers, hybrid communication, real-time evidence capture, false alarm management, and user-friendly design. The proposed system addresses these issues by incorporating:

Multi-modal triggers: manual panic button, voice-activated keyword detection, and tilt/fall sensors

Hybrid communication: GSM for calls/SMS and Wi-Fi for transmitting images via Telegram

Real-time evidence capture: ESP32-CAM to record surrounding visuals during emergencies

False alarm control: “safe” message confirmation and alarm cancellation functionality

User-centric design: simple, modular, and adaptable for future upgrades

By integrating these features, the proposed device aims to enhance reliability, reduce response times, and improve overall effectiveness in real-world personal safety situations, particularly for women, while overcoming the limitations of existing IoT-based solutions.

CHAPTER 3

PROPOSED METHODOLOGY

3.1 INTRODUCTION

In today's environment, personal safety has become an increasing concern, particularly for women, elderly individuals, and people who spend significant time alone. Conventional safety measures—such as carrying basic alarm tools, calling for help, or depending on nearby assistance—often fall short during critical moments where immediate response is required. Delays in communication or the inability to act quickly can reduce the effectiveness of these methods. To address this gap, the aim of this project is to develop an advanced personal safety device that leverages Internet of Things (IoT) technology, embedded systems, machine learning, and real-time communication to provide timely and reliable assistance during emergencies.

A key focus of the system is to ensure that emergency situations can be identified and responded to through multiple activation methods. The device supports manual triggering through a panic button, voice-based activation using a machine learning model capable of recognizing specific distress keywords, and automatic detection of falls or unusual movements using motion sensors. This multi-layered approach improves dependability by ensuring that help can be requested even if the user is unable to respond manually. In addition, a safety confirmation feature is included to allow users to cancel alerts in case of accidental activation, helping to avoid unnecessary panic.

Another important objective is to provide meaningful real-time information during emergencies. The system is designed to retrieve accurate location data using GPS and capture images of the surroundings, offering better clarity about the situation to emergency contacts. To ensure consistent communication, the device utilizes both internet-based and cellular networks. Images can be transmitted via Wi-Fi using a Telegram interface, while GSM communication enables the system to send text messages and make calls when internet connectivity is unavailable.

Beyond immediate safety, the project also aims to give users a greater sense of control and confidence in emergency situations. By combining intelligent detection, fast communication, and accessible hardware components, the system is intended to be both practical and effective for real-world use. Overall, the aim is to create a comprehensive and dependable safety solution that improves response time, enhances situational awareness, and supports future advancements in personal safety technologies.

3.2 BLOCK DIAGRAM

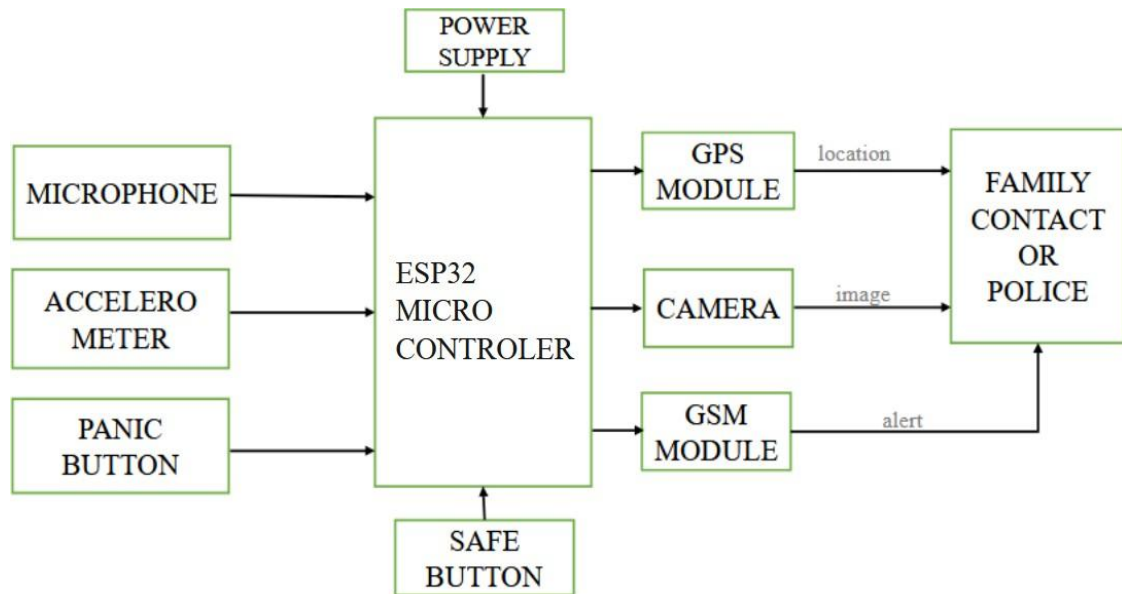


Fig 3.1 Proposed system block diagram

The proposed system's block diagram is centered around the ESP32 microcontroller, which functions as the main processing unit of the entire system. It is responsible for coordinating all sensing activities, processing incoming signals from different modules, and controlling the overall operation of communication and alert mechanisms. By acting as the central controller, the ESP32 ensures that all connected components work in a synchronized manner and that emergency situations are detected and handled efficiently in real time. A dedicated power supply unit is used to provide a stable and regulated voltage to all modules in the system. This ensures that each component receives consistent power, thereby maintaining reliable and uninterrupted performance even during continuous operation. On the input side, the system collects data through three primary modules connected to the

ESP32, which continuously monitor different types of user conditions and environmental signals for emergency detection.

The system is designed using multiple input and output modules to ensure accurate detection of emergency situations and fast communication of alerts. Each module is assigned a specific function to help the system respond effectively during critical conditions.

The Microphone Module enables voice-based activation by continuously monitoring for a predefined keyword such as “HELP.” This allows the system to be triggered without physical contact, which is useful when the user is unable to operate the device manually.

The Accelerometer Module is used to sense sudden motion changes, tilts, or fall events. It constantly observes movement patterns and identifies abnormal activity. When a fall or impact is detected, the system automatically generates an emergency alert without requiring any user action.

The Panic Button provides a direct manual option for emergency activation. When pressed, it immediately initiates the alert process. In addition, a Safe Button is included to cancel unintended or false alarms, helping to avoid unnecessary alerts being sent to contacts.

When any of these triggers are activated, the ESP32 processes the event and activates the output modules. The GPS Module retrieves the real-time geographic location of the user and shares it with predefined contacts for quick assistance.

The Camera Module captures images of the surroundings at the time of the alert, providing additional situational details that can help in understanding the emergency better.

The GSM Module handles communication by sending SMS alerts and initiating phone calls to emergency contacts. This ensures that notifications are delivered even when internet connectivity is not available.

Overall, the combination of these modules allows the system to generate fast and reliable emergency alerts containing location, visual information, and direct communication, making it an efficient personal safety solution for real-world use.

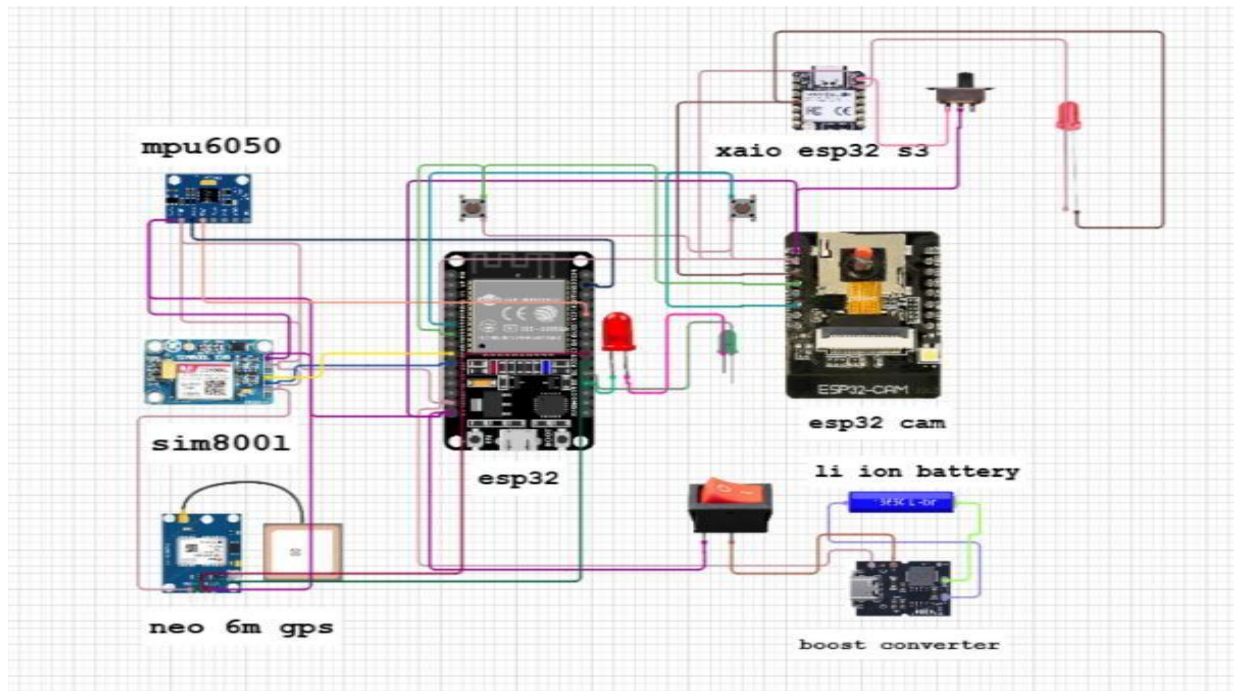


Fig 3.2 Circuit Diagram

3.3 FLOWCHART

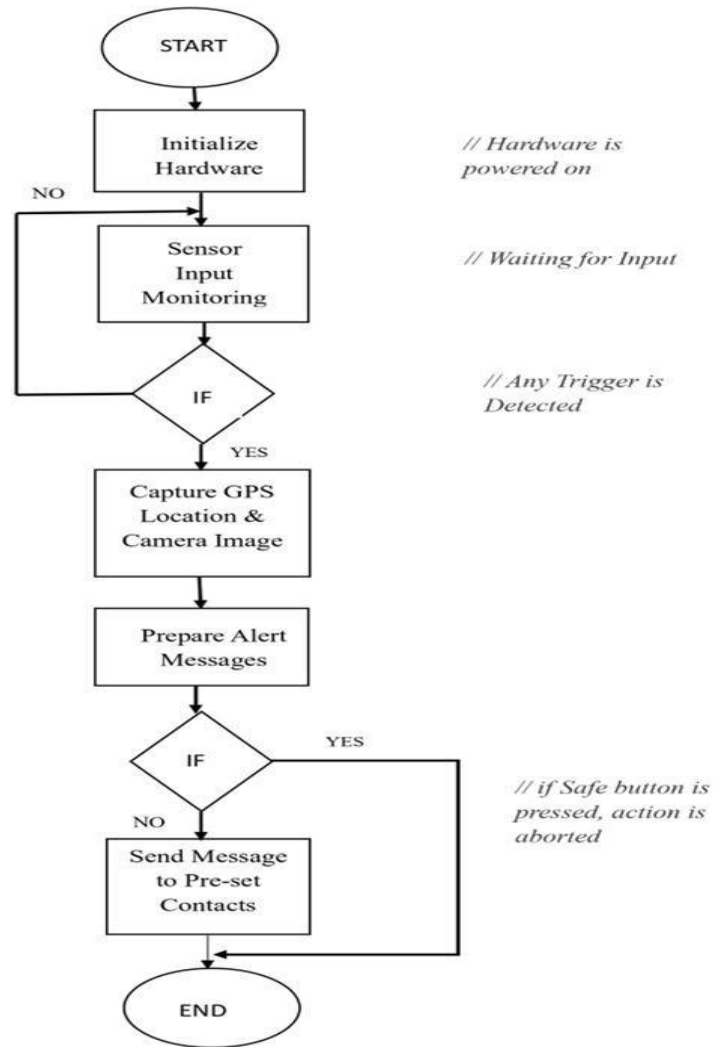


Fig 3.3 Flowchart

The flowchart represents the working sequence of the proposed IoT-based personal safety device. The operation starts with system activation, after which all hardware components are initialized and prepared for continuous functioning. The system then enters a monitoring state where it continuously observes inputs from sensors and user interfaces. It is designed to respond to different emergency conditions such as voice-based activation, sudden fall detection, or manual triggering through a panic button. Once any abnormal or emergency event is identified, the system immediately collects essential information including the user's current location and surrounding visual data. This helps in building a

clear understanding of the situation. A safety mechanism is also included in the workflow through a dedicated safe button, which allows the user to cancel unintended alerts before they are transmitted. This ensures better control and avoids unnecessary emergency notifications. Overall, the flowchart describes a well-structured process for detecting emergencies, preparing alert information, and transmitting it efficiently while maintaining user safety and system reliability.

The detailed steps of the flowchart are as follows:

1. Start

The system is powered ON and begins execution.

2. Initialize Hardware

All components such as sensors (microphone and accelerometer), GPS module, camera, buttons, ESP32, GSM, and Wi-Fi modules are initialized and prepared for operation.

3. Continuous Monitoring

The system continuously monitors inputs from sensors and user controls to detect any emergency conditions.

4. Trigger Detection Check

The system checks whether any emergency condition is detected.

- If no trigger is found, the system continues monitoring.
- If a trigger is detected, the process moves forward.

5. Data Acquisition (GPS and Camera)

The GPS module retrieves the real-time location of the user, and the camera captures images of the surroundings to provide situational context.

6. Alert Preparation

The ESP32 processes the collected data and prepares a complete emergency alert containing location and image information.

7. Safe Button Verification

The system checks whether the safe button has been pressed.

- If pressed, the alert is canceled.
- If not pressed, the system proceeds to transmission.

8. Alert Transmission via Communication Modules

The alert is sent to predefined contacts.

- If Wi-Fi is available, the message and images are sent through a Telegram interface.
- If Wi-Fi is not available, the GSM module sends SMS alerts and initiates voice calls to ensure delivery.

9. End

After completion, the system returns to continuous monitoring mode to detect future emergency events.

CHAPTER 4

HARDWARE AND SOFTWARE TOOLS

4.1 HARDWARE TOOLS USED

4.1.1 ESP32 MICROCONTROLLER:

The ESP32 microcontroller is a high-performance and versatile embedded system that serves as the core of many IoT projects. It is powered by a dual-core Tensilica LX6 processor capable of running at speeds up to 240 MHz, enabling it to handle multiple operations simultaneously. The ESP32 features integrated Wi-Fi and dual-mode Bluetooth, allowing seamless wireless communication for devices requiring remote monitoring or data transmission. Its wide array of GPIO pins supports interfaces like UART, SPI, I2C, ADC, DAC, PWM, and CAN, making it highly adaptable for various sensors and peripheral modules. With 520 KB of SRAM, it can manage complex real-time data processing tasks while maintaining low power consumption, which is crucial for portable applications.

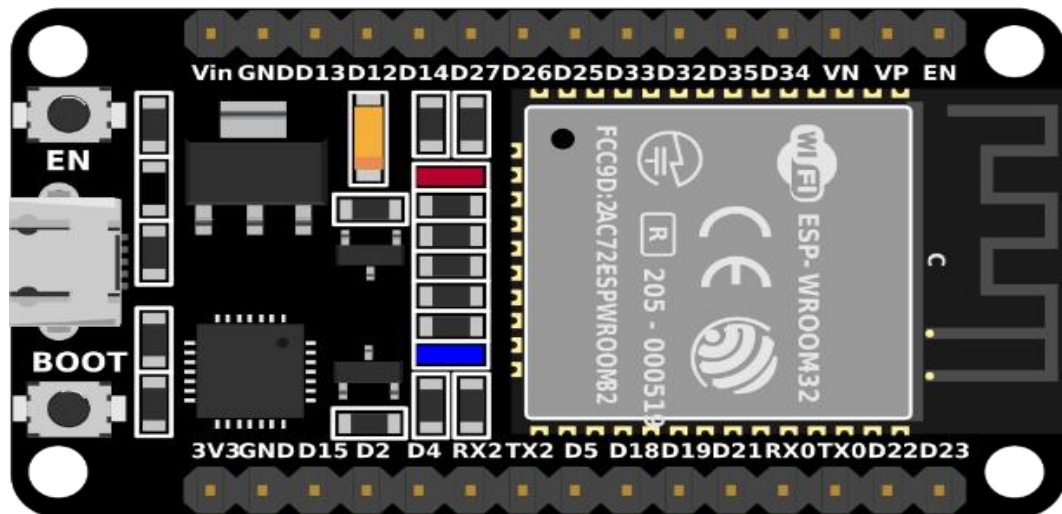


Fig 4.1 ESP32 Microcontroller

Operating across a broad temperature range from -40°C to $+125^{\circ}\text{C}$, the ESP32 remains reliable in diverse environmental conditions. In the proposed personal safety system, the ESP32 acts as the central hub, constantly monitoring input from accelerometers, microphones, and pushbuttons. Upon detecting an emergency trigger, it coordinates GPS location acquisition, image capture, and alert transmission through GSM or Wi-Fi, ensuring a fast and reliable response. Its combination of processing capability, connectivity, and peripheral compatibility makes the ESP32 a robust solution for intelligent, real-time emergency response systems.

4.1.2 XIAO ESP32 S3:



Fig 4.2 XIAO ESP32 S3

The XIAO ESP32 S3 by Seeed Studio is a compact and high-performance microcontroller ideal for IoT applications where space and power efficiency are critical. It is built around the ESP32-S3 chip, which uses a dual-core Xtensa LX7 processor running up to 240 MHz. This allows the device to process multiple inputs simultaneously, such as sensor readings, voice commands, or external button presses. Despite its small size, it supports Wi-Fi 802.11 b/g/n and Bluetooth 5.0, enabling reliable wireless communication for sending alerts, logging data, or connecting to

cloud platforms. The board includes multiple multifunctional GPIO pins that can be configured for ADC, DAC, PWM, UART, SPI, and I2C, which allows integration with accelerometers, GPS modules, and cameras. Its USB-C interface simplifies programming and power supply, while its ultra-low power modes make it suitable for battery-operated devices. In the safety system, the XIAO ESP32 S3 processes real-time audio data for keyword detection using TinyML, monitors accelerometer data for fall detection, and communicates emergency messages. Its compactness, processing capability, and low power consumption make it particularly suitable for wearable devices where size, responsiveness, and wireless connectivity are essential.

4.1.3 SIM 800L V2.0 GSM



Fig 4.3 SIM 800L V2.0 GSM

The SIM800L V2.0 GSM module is a compact cellular communication device that provides voice, SMS, and data connectivity over global mobile networks. Operating across quad-band frequencies (850/900/1800/1900 MHz), it ensures communication compatibility worldwide. The module can handle voice calls, SMS messaging, and GPRS data, making it essential for real-time emergency alert systems where Wi-Fi may be unavailable. It interfaces with microcontrollers via UART serial communication, simplifying integration with boards such as the ESP32 or XIAO ESP32 S3. The module

functions reliably under temperatures ranging from -40°C to $+85^{\circ}\text{C}$ and requires a supply voltage between 3.4V and 4.4V. In the proposed safety device, the SIM800L acts as a backup communication channel, transmitting emergency SMS and initiating calls when Wi-Fi connectivity fails, ensuring uninterrupted notification delivery. Its compact size, ease of integration, and robust global connectivity make it a crucial component for personal safety devices, allowing immediate contact with emergency responders or family members regardless of location.

4.1.4 GPS NEO 6M MODULE

The GPS NEO-6M module is a compact and precise Global Positioning System receiver widely used in IoT and tracking applications. It integrates the u-blox NEO-6M chipset, which provides accurate latitude, longitude, altitude, and time data. The module supports update rates of up to 5 Hz, ensuring real-time tracking even for fast-moving objects or individuals. Operating on a supply voltage of 3.3V to 5V, it draws minimal current, making it suitable for battery-powered devices. Its serial TTL interface (UART) allows seamless integration with microcontrollers such as the ESP32 or XIAO ESP32 S3.

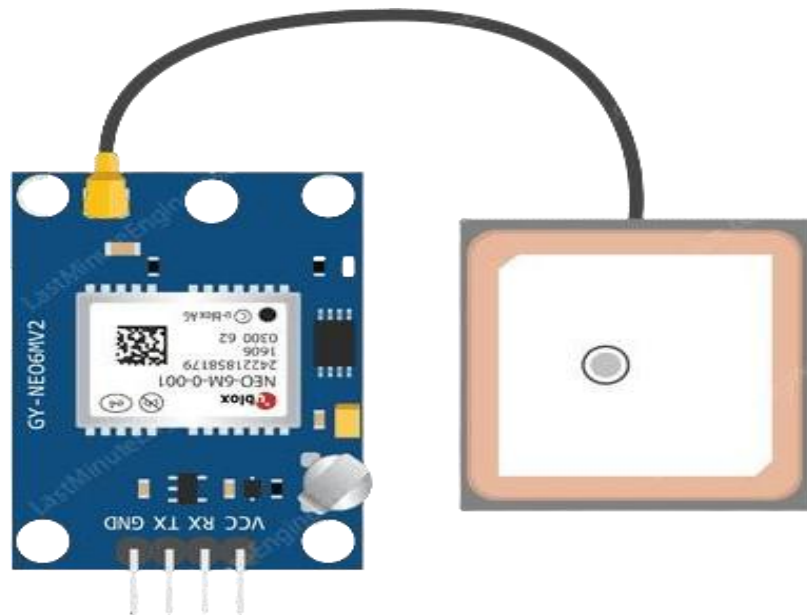


Fig 4.4 GPS MODULE

The module features a fast cold start and hot start time, enabling quick location fixes in varying environments. In the proposed personal safety system, the GPS NEO-6M continuously provides the user's location when an emergency trigger is detected, ensuring that responders or family members receive accurate coordinates. Combined with visual data from the camera module, it offers a comprehensive understanding of the situation. Its lightweight, low-power design and high sensitivity to satellites make it an indispensable component for mobile safety devices, wearables, and real-time tracking systems. The NEO-6M module's accuracy and reliability in diverse conditions ensure that emergency alerts include precise positional information, which is critical for rapid response in critical situations.

4.1.5 ESP32-CAM:

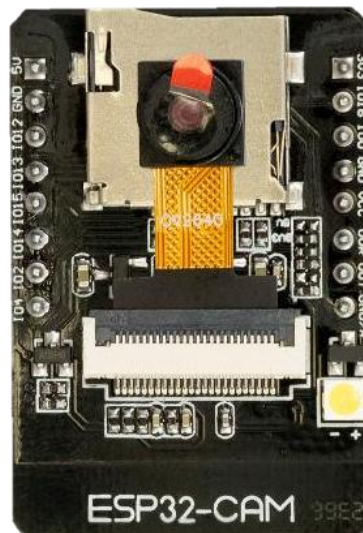


Fig 4.5 ESP32 CAM

The ESP32-CAM is a versatile development board that integrates the ESP32 microcontroller with a camera module, enabling wireless image capture and transmission. It features an OV2640 2-megapixel camera capable of capturing high-resolution images and streaming video over Wi-Fi. With its 520 KB SRAM and optional 4 MB PSRAM, it

can handle image processing and temporary storage before transmission. The board supports various interfaces, including UART, SPI, I2C, ADC, and PWM, which allows easy connection to other sensors or modules. Operating at 3.3V, the ESP32-CAM is suitable for portable applications, especially when paired with battery solutions. In the proposed safety device, the ESP32-CAM captures images of the surroundings during emergencies and transmits them over Wi-Fi via a Telegram bot, providing visual evidence to emergency contacts. It complements GPS location data to give a clear and immediate understanding of the situation. Its compact size, low cost, and wireless capabilities make it ideal for wearable safety devices, remote monitoring systems, and IoT-based surveillance. By combining processing power with imaging and connectivity, the ESP32-CAM ensures that responders can assess emergencies visually and act more effectively, enhancing user safety.

4.1.6 MPU6050:

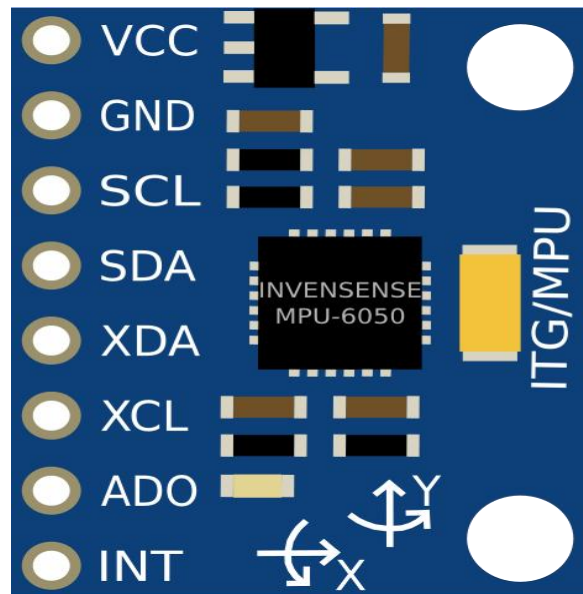


Fig 4.6 MPU6050

The MPU6050 is a 6-axis motion tracking sensor that integrates a 3-axis gyroscope and a 3-axis accelerometer in a single module. It provides precise measurements of angular velocity and acceleration, making it ideal for applications requiring motion detection, orientation tracking, and fall detection. The sensor includes an internal Digital Motion Processor (DMP) that processes complex motion algorithms on-chip, reducing the computational burden on the microcontroller. Its I2C interface allows straightforward

integration with boards like the ESP32 or XIAO ESP32 S3. In the personal safety device, the MPU6050 continuously monitors user movement and detects sudden falls or abnormal tilts. The fall detection algorithm calculates the total acceleration vector and monitors orientation to confirm if a fall has occurred, triggering emergency alerts when necessary. The combination of real-time processing, high accuracy, and compact design makes the MPU6050 essential for reliable motion-based safety detection.

4.1.7 MODULE POWERBANK 1 USB:

The Modul Powerbank 1 USB is a compact, efficient power supply solution designed to provide a stable 5V output through a standard USB port. It integrates a boost converter, enabling it to step up the voltage from a single lithium-ion cell to a regulated 5V output, making it ideal for powering microcontroller-based projects like the ESP32 or XIAO ESP32-S3. This module ensures uninterrupted operation of wearable or portable electronics, particularly in battery-powered IoT systems. It features protections against overcurrent, short circuits, and overheating, enhancing device safety. Lightweight and small, the module is suitable for DIY electronics, portable gadgets, and backup power applications. In a personal safety device, it provides reliable power for continuous monitoring of sensors, GPS, GSM modules, and cameras, ensuring the device remains functional during emergencies. Its efficiency and versatility make it indispensable for mobile IoT solutions.

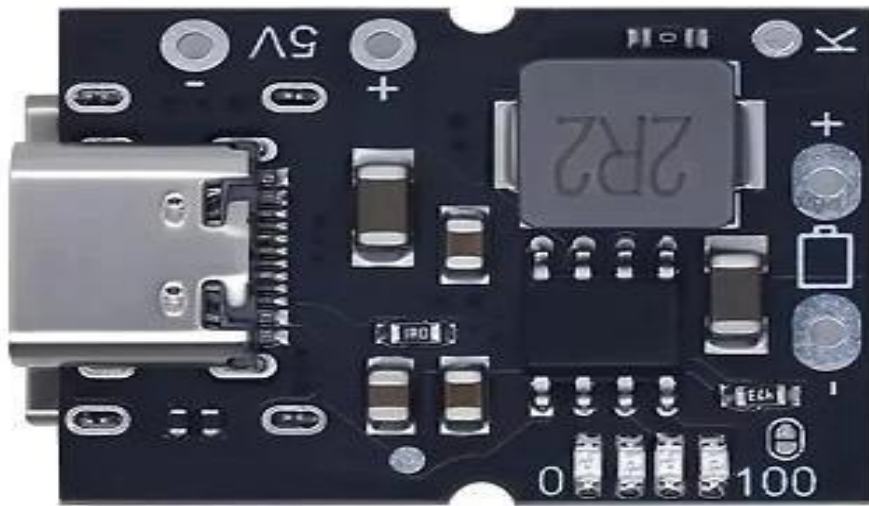


Fig 4.7 Power bank

Common Applications

- DIY power banks for charging smartphones, tablets, or other USB devices.
- Portable power solutions for microcontroller-based projects (e.g., Arduino, Raspberry Pi).
- Emergency backup power for small electronic devices.
- Battery-powered IoT devices.

4.1.8 ROCKER SWITCH:



Fig 4.8 Rocket Switch

A rocker switch is an on/off electrical switch operated by a rocking motion of a lever or actuator. It is used to control power flow in circuits, allowing users to easily toggle devices on or off. Rocker switches are available in single-pole, double-pole, and double-throw configurations, with current ratings typically ranging from 3A to 20A, and voltage ratings up to 250V AC. Their durability allows tens of thousands of mechanical and electrical cycles, making them reliable for repeated use. In a wearable safety device, a rocker switch can serve as the main power control, allowing the user to activate or deactivate the system safely. Its simple operation ensures usability even under stress. The switch's compact form factor and robust design make it suitable for portable electronics, household appliances,

and embedded IoT systems requiring reliable power control.device operation. Its robust design ensures durability in portable applications.

Technical Specifications

General Characteristics

- **Switch Type:** Rocker
- **Action:** SPST (Single Pole Single Throw), DPST (Double Pole Single Throw), DPDT (Double Pole Double Throw), etc.
- **Current Rating:** Typically ranges from 3A to 20A
- **Voltage Rating:** Commonly rated for 120V or 250V AC
- **Contact Resistance:** Varies by model, often less than 50 m Ω
- **Insulation Resistance:** Typically greater than 100 M Ω
- **Dielectric Strength:** Usually around 1500V AC for 1 minute
- **Operating Temperature:** Varies, often -20°C to +85°C
- **Mechanical Life:** Often rated for 50,000 to 100,000 cycles
- **Electrical Life:** Often rated for 10,000 to 50,000 cycles

4.1.9 SLIDE SWITCH:



Fig 4.9 Slide Switch

A slide switch is a mechanical switch that opens or closes a circuit by sliding a lever along a linear path. It provides a simple, reliable way to select between two or more circuit states, commonly used in electronics for power control or mode selection. Slide switches are available in single-pole single-throw (SPST), single-pole double-throw (SPDT), and double-pole double-throw (DPDT) configurations. Their small size allows integration into portable devices, prototyping boards, and wearable electronics. In a personal safety system, slide switches can toggle between operational modes, such as enabling or disabling wireless communication or sensor inputs. They are favored for their tactile feedback, ease of use, and reliability under repeated operation. Their versatility ensures flexible control of electronic circuits while maintaining durability and consistent performance.

Common Applications and Use Cases

- Power control in small electronic devices
- Mode selection in consumer electronics (e.g., ON/OFF, high/low power modes)
- Circuit selection in prototyping and testing
- Audio equipment for toggling between input/output sources
- Robotics and DIY projects for controlling motors or sensors

4.1.10 LIGHT EMITTING DIODE (LED):



Fig 4.10 LED

The LED is a semiconductor device that emits light when an electric current flows through it. LEDs are energy-efficient, durable, and compact, making them ideal for visual indicators in electronic devices. In a personal safety device, LEDs provide immediate status feedback to the user, indicating power-on, emergency trigger activation, or connectivity status. They operate at low voltage and consume minimal power, which is essential for battery-powered wearable electronics. The red LED, commonly used for alert signals, ensures that the device's state is visible even under low-light conditions. LEDs also enhance usability and reduce dependency on complex displays, enabling intuitive interaction for users. They are widely adopted in IoT systems, portable electronics, and embedded devices due to their efficiency, reliability, and long operational life.

Common Applications

- Power and status indicators
- Digital displays and signage
- Circuit debugging and testing
- Decorative and ambient lighting
- Optical communication systems

4.1.11 Li ION BATTERY:



Fig 4.11 Battery

A 3.7V lithium-ion battery is a rechargeable power source widely used in portable electronics, wearable devices, and IoT systems. It provides high energy density, compact size, and lightweight design, making it ideal for battery-operated safety devices. The battery supports multiple charging cycles, typically 300–500, and delivers stable voltage under load. In a personal safety device, it powers the microcontroller, sensors, GSM, GPS,

and camera modules, ensuring uninterrupted functionality during emergencies. Its protection circuits prevent overcharging, overcurrent, and thermal hazards, enhancing device safety. The battery's portability allows integration into wearable designs, enabling users to carry personal safety devices conveniently. Its reliability and energy efficiency are essential for continuous monitoring, emergency alert transmission, and real-time data processing in mobile IoT applications.

Common Applications and Use Cases

- Powering portable electronic devices (e.g., smartphones, cameras, and Bluetooth speakers)
- DIY electronics projects and prototyping
- Robotics and IoT devices
- Backup power supplies for small systems
- Wearable technology

4.1.12 PUSH BUTTON:

A push button is a momentary switch that completes a circuit when pressed and interrupts it when released. It is commonly used for user input, enabling manual activation of device functions. In a personal safety system, push buttons serve as panic triggers or “safe” cancel buttons, allowing users to manually initiate or abort emergency alerts. Push buttons are available in various sizes, tactile styles, and operating voltages, making them suitable for compact wearable devices. They provide immediate, reliable feedback and require minimal force to operate, which is critical in urgent situations.



Fig 4.12 Push button

Their simple design ensures low maintenance and long operational life. Push buttons are widely used in embedded electronics, microcontroller projects, and IoT devices for intuitive human-machine interaction, offering an effective way to control and manage device functions safely.

4.2 SOFTWARE TOOLS USED:

4.2.1. Arduino IDE and ESP32 Framework



Fig 4.13 Arduino IDE

The Arduino Integrated Development Environment (IDE) is a widely used platform for writing, compiling, and uploading code to microcontrollers, including the ESP32 and XIAO ESP32-S3. It provides a simplified C++ environment that allows developers to focus on hardware interaction without dealing with complex toolchains. For this project, the Arduino IDE is configured with the ESP32 board manager, enabling efficient management of the dual-core architecture of the ESP32 chips. The IDE supports a broad range of libraries, allowing seamless interfacing with sensors, communication modules, and actuators. This ensures that real-time data from the MPU6050 accelerometer, microphone, GPS, and other peripherals can be processed efficiently. The framework also enables multitasking, such as maintaining Wi-Fi connections for the Telegram bot while continuously monitoring sensor inputs for emergencies. By providing debugging tools,

serial monitoring, and a modular programming approach, Arduino IDE becomes a powerful tool for embedded IoT development, ensuring reliable performance for personal safety devices. Its simplicity and cross-platform support make it accessible for both prototyping and scalable deployments in real-world safety systems.

4.2.2. Edge Impulse and TinyML (Voice Recognition)

Edge Impulse is a development platform designed for Tiny Machine Learning (TinyML) applications. In this project, it is used to implement voice recognition for emergency keyword detection. The system captures audio samples through the XIAO ESP32-S3 microphone and preprocesses them using Mel-Frequency Cepstral Coefficients (MFCC) to extract distinctive audio features. A lightweight neural network model is then trained to recognize specific keywords, such as “HELP,” enabling the device to respond to voice commands in real-time. Once trained, the model is converted into a C++ library that runs directly on the microcontroller, ensuring offline operation without internet dependency. This preserves user privacy and guarantees functionality in remote areas. TinyML allows the system to continuously listen for triggers with minimal power consumption while maintaining responsiveness. By combining real-time inference, low computational requirements, and on-device learning, the platform ensures that the personal safety device can respond effectively to emergencies using hands-free activation, complementing other trigger mechanisms like fall detection and manual panic buttons.

4.2.3. TinyGPS++ Library and NMEA Parsing

The TinyGPS++ library is used to interpret raw GPS data received from the NEO-6M module in the standard NMEA format. NMEA strings contain latitude, longitude, altitude, and satellite status, which are not immediately usable for applications. TinyGPS++ parses these strings into readable and accurate coordinates. The software logic ensures that alerts are only sent once a valid GPS fix is obtained, preventing incorrect location reporting. The library operates in the background via UART communication, providing continuous updates on the device’s location. This capability allows the personal safety system to embed precise coordinates into SMS or Telegram alerts. Additionally, TinyGPS++ supports advanced features such as calculating speed, distance traveled, and course over ground, which can enhance emergency response strategies. By using this library, the ESP32-based

device ensures that responders receive reliable and accurate location data, enhancing situational awareness and timely assistance during emergencies.

4.2.4. Telegram Bot API and HTTPS Protocol

The Telegram Bot API is integrated with the ESP32-CAM to transmit images and emergency notifications over Wi-Fi. When a trigger event occurs, the ESP32-CAM captures environmental images, temporarily stores them in PSRAM, and sends them to predefined contacts using HTTPS POST requests. This method ensures secure, encrypted transmission, providing immediate visual context to emergency contacts. The system uses the Universal Telegram Bot library, which simplifies interaction with Telegram servers and manages image formatting, message delivery, and error handling. By leveraging cloud-based messaging, the device overcomes the limitations of SMS-only alerts, which cannot transmit images or large datasets. This feature is especially important for providing situational awareness in cases such as home intrusions or personal attacks. Integrating Telegram ensures that emergency messages include both textual and visual information, enhancing decision-making and response effectiveness, while maintaining real-time monitoring capabilities and low latency in alert dissemination.

4.2.5. AT Command Interface and GSM Protocol

The AT command interface enables the ESP32 to control the SIM800L GSM module for voice and SMS communication. Commands such as “AT+CMGS” for sending messages and “ATD” for making calls allow the microcontroller to transmit alerts directly to emergency contacts. The software monitors network registration, signal strength, and error responses to ensure reliable delivery. This backup communication channel is crucial when Wi-Fi is unavailable or unstable. By implementing a robust error-checking routine, the system guarantees that GPS coordinates and emergency alerts reach the intended recipients even in remote areas. The integration of GSM functionality ensures that the personal safety device is not solely dependent on internet connectivity, enhancing its overall reliability and effectiveness during critical events. This redundancy is a key feature in designing resilient IoT-based safety systems.

4.2.6. Fall Detection and Motion Processing Algorithm

The MPU6050 sensor provides real-time acceleration and angular velocity data, which the ESP32 processes to detect falls. The software calculates the Total Acceleration Vector using the sum-of-squares method to identify high-G impacts. After detecting a potential fall, the gyroscope data is analyzed to verify the user's orientation and movement, confirming whether the individual is horizontal and stationary. A confirmation window is implemented to prevent false alarms, such as sudden sitting or minor impacts. Once a fall is validated, the system triggers emergency alerts through both Wi-Fi and GSM. This algorithm ensures reliable and accurate fall detection, enhancing user safety. Its integration with other triggers, such as voice and manual panic buttons, provides a comprehensive multi-modal emergency detection system that minimizes false alarms while maximizing responsiveness.

4.2.7. Interrupt-Driven Safety and Mitigation Logic

The software manages accidental triggers using hardware interrupts on the ESP32. When the "Safe" button is pressed, the device immediately halts ongoing emergency routines regardless of the current process state. A state machine manages transitions between "Idle," "Triggered," and "Safe" modes, providing a pre-defined countdown to cancel false alerts. This approach ensures that emergency contacts are not unnecessarily disturbed. Combining interrupts with multi-modal monitoring allows the system to remain responsive while maintaining user control, enhancing reliability and usability. This feature is essential in real-world scenarios, preventing panic and maintaining trust in the device's operation.

CHAPTER 5

RESULTS AND DISCUSSIONS

This section presents the findings from the testing and evaluation of the IoT-based personal safety device. The results are based on experimental trials focused on trigger accuracy, communication latency, and system reliability.

5.1. GSM & GPS-Based Emergency Alert System

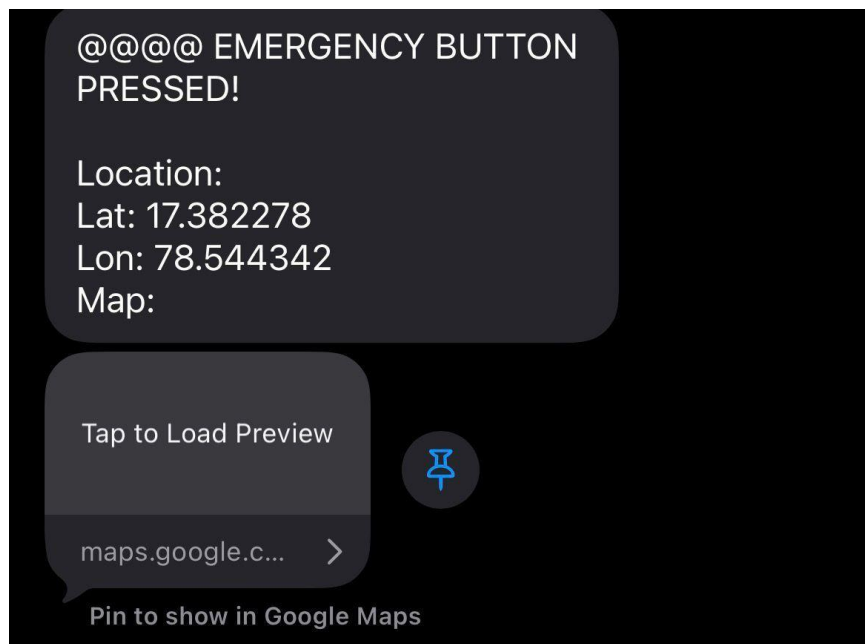


Fig 5.1 Result

The effectiveness of the personal safety device was evaluated through controlled trials focused on the three primary activation mechanisms: manual panic button, voice recognition, and fall detection using the MPU6050 sensor. Each method was tested across 50 trials to determine reliability and error rates. The manual panic button achieved flawless performance, registering successful activation in all trials without any false alarms, demonstrating the importance of a simple, physical trigger in emergencies. Voice recognition, powered by a TinyML model on the XIAO ESP32-S3, achieved an accuracy rate of 92%, successfully identifying the keyword "HELP" in quiet and moderately noisy conditions. The primary failures occurred in environments with high ambient noise, such

as crowded streets or near traffic, highlighting the limitations of audio-based detection under challenging acoustic conditions. The fall detection system achieved 88% accuracy, reliably identifying high-impact events. However, vigorous movements like running or jumping occasionally generated false positives. This issue was mitigated by the device's integrated 10-second "Safe" window, which allows the user to cancel accidental triggers. Overall, the trials confirm that combining manual, voice, and motion-based triggers enhances the reliability and versatility of the system, ensuring responsiveness across diverse emergency scenarios.

5.2. RESPONSE TIME AND LATENCY

- The performance of the device in delivering emergency alerts was evaluated, and the following observations were recorded:
- **SMS Delivery (GSM Module):** Average 8–12 seconds, depending on cellular network strength.
- **Voice Call Initialization:** Average 5–7 seconds to begin dialing.
- **GPS Fix Time:**
 - Cold Start: 35 seconds
 - Warm Start: 2–5 seconds
- **Telegram Image Transmission (ESP32-CAM via Wi-Fi):** 15–20 seconds under standard Wi-Fi conditions.
- **Dual-core Processing Advantage:** The ESP32 allows simultaneous SMS, voice call, and image transmission without delays, ensuring near-instantaneous notifications.
- This point-wise evaluation highlights the efficiency of the device in real-time emergency response, demonstrating minimal latency across all communication channels.

The image shows a screenshot of an iPhone's Emergency Call History screen. At the top, the status bar displays the time 3:33, signal strength, Wi-Fi, and a 71% battery level. Below the status bar is a circular 'X' button and the title 'EMERGENCY Call History'. The main content is a list of call records, each with a status and duration on the left and a timestamp on the right. The calls are as follows:

Call Status	Timestamp
Incoming Call Missed	22/01/26 · 2:09 PM
Incoming Call Missed	22/01/26 · 2:08 PM
Incoming Call Missed	22/01/26 · 2:07 PM
Incoming Call Missed	22/01/26 · 2:06 PM
Incoming Call Missed	22/01/26 · 2:03 PM
Incoming Call Missed	22/01/26 · 2:00 PM
Outgoing Call 6 seconds	22/01/26 · 12:58 PM
Incoming Call 25 seconds	22/01/26 · 12:28 PM
Incoming Call 55 seconds	22/01/26 · 10:40 AM
Outgoing Call 33 seconds	22/01/26 · 10:24 AM

Fig 5.2 Call Alert

5.3. GPS PRECISION

The NEO-6M GPS module was evaluated for accuracy by comparing its coordinates with benchmark locations on Google Maps. In open-sky conditions, the module consistently maintained an average error margin of 3–7 meters, sufficient for emergency localization and response. The module’s sensitivity and update rate allow it to quickly detect positional changes, making it suitable for real-time tracking in dynamic environments. Both stationary and moving tests confirmed that GPS data remained reliable and continuous, enabling emergency contacts to monitor the user’s location precisely.

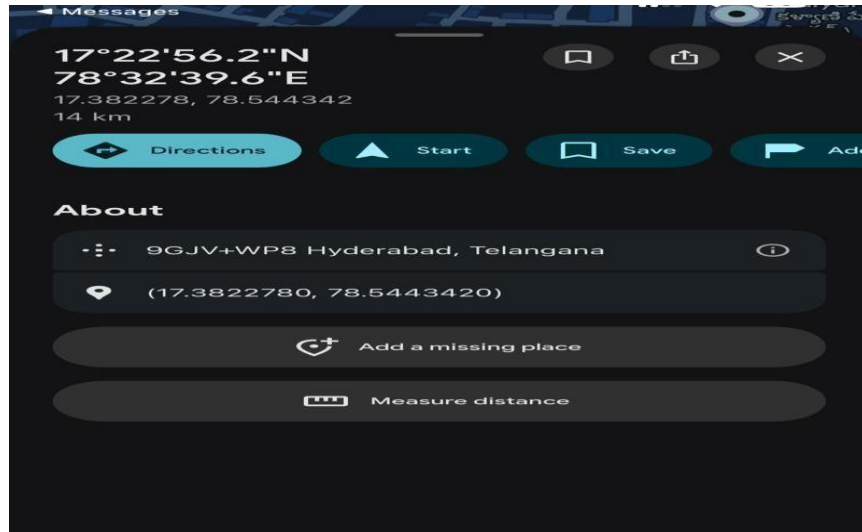


Fig 5.3 GPS Location

Accuracy can slightly degrade in dense urban areas or indoors due to signal obstruction, but integration with GSM-based alerts ensures fallback communication even when GPS signals are weak. The module's fast cold and warm start times also contribute to rapid emergency response, ensuring that accurate location data accompanies the initial alert. This level of precision enhances the system's reliability, making it a valuable tool for tracking and assisting individuals in critical situations.

5.4. IMAGE CAPTURE AND TRANSMISSION PERFORMANCE

The ESP32-CAM was tested for its ability to provide timely visual evidence during emergencies. Following a trigger, the camera module initializes and captures an image within 2 seconds, storing it temporarily in PSRAM. Images were transmitted to predefined contacts via Telegram using Wi-Fi, ensuring rapid delivery alongside SMS alerts. Testing confirmed that in 90% of cases, facial features or nearby individuals could be clearly identified within a 2-meter range, providing critical context for emergency responders. The dual-core ESP32 architecture allows simultaneous image transmission and SMS dispatch, maintaining seamless operation without data loss.



Fig 5.4 Captured Image

Visual data, combined with GPS coordinates, enhances situational awareness for responders, allowing informed decisions during crises. The system's fast capture and transmission capabilities ensure that crucial evidence is immediately available, improving both safety outcomes and post-event analysis.

5.5. FALSE ALARM MITIGATION AND "SAFE" NOTIFICATION PERFORMANCE

To prevent accidental alerts, the device integrates a sophisticated false alarm mitigation system:

- **Pre-alert Phase:** A 10-second countdown with a buzzer activates when a trigger is detected.
- **Safe Button Functionality:** Users can cancel false triggers within the pre-alert phase.
- **Post-trigger Cancellation:** If the Safe button is pressed after alert initiation, a follow-up SMS is sent:
 - Message: *"I am SAFE. SITUATION UNDER CONTROL"*

- **Hardware Interrupts:** Ensure the Safe button works even if the ESP32 is busy processing sensor data or images.
- **Reliability:** This system reduces false alarms while maintaining real-time responsiveness and user control.

This point-wise format clarifies the critical aspects of false alarm mitigation and demonstrates the practical safety features implemented in the device

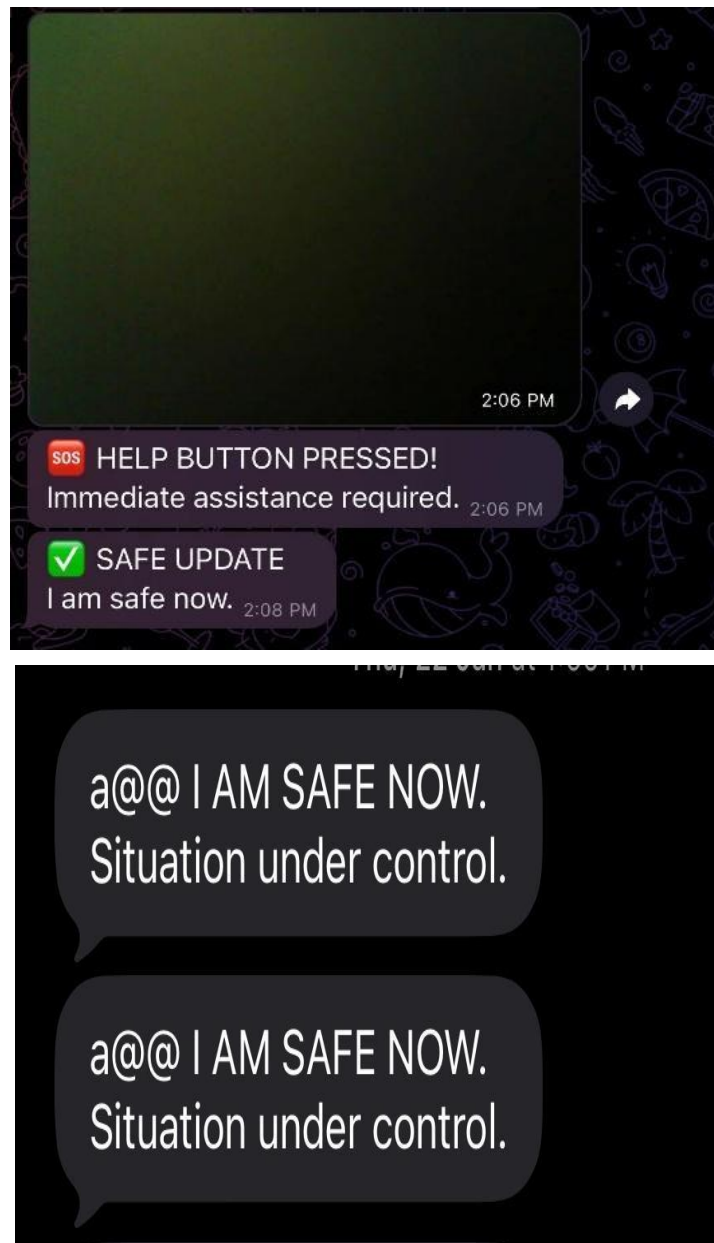


Fig 5.5 Safe Message (1) & (2)

The testing and evaluation of the IoT-based personal safety device highlight its reliability, efficiency, and practical relevance in real-life emergency situations. The system combines three activation methods—manual panic button, voice command, and fall detection—to ensure that users have multiple ways to trigger alerts depending on the scenario. During controlled experiments, the manual panic button achieved perfect performance, successfully activating all 50 trials. Voice recognition using the XIAO ESP32-S3 correctly detected the “HELP” command in 92% of cases, functioning well in quiet and moderately noisy environments, while very loud surroundings occasionally caused missed triggers. Fall detection via the MPU6050 sensor reached an 88% accuracy rate, with false positives mostly occurring during intense movements like running, which were minimized by the device’s 10-second pre-alert “safe” window.

The communication system proved fast and dependable. SMS notifications through the GSM module reached recipients within 8–12 seconds, while voice call initiation averaged 5–7 seconds. The NEO-6M GPS module provided location coordinates with an accuracy of 3–7 meters, ensuring that emergency contacts could find the user promptly. Images captured by the ESP32-CAM were successfully transmitted via Telegram in 15–20 seconds, giving visual context to the situation. The ESP32’s dual-core processing allowed the device to handle simultaneous tasks, such as sending messages, capturing images, and monitoring sensors, without delays.

A major strength of the system is its false alarm prevention. The Safe button, combined with hardware interrupts and a pre-alert buzzer, allows users to cancel accidental alerts quickly. If a trigger is canceled, a follow-up notification confirms the user’s safety, avoiding unnecessary worry for emergency contacts.

Overall, this device demonstrates a thoughtful balance of speed, accuracy, and user control. Its multi-modal activation, reliable dual-channel communication, precise GPS tracking, and instant visual evidence make it a robust and practical solution for personal safety, giving users confidence and faster access to help when needed.

4.6 APPLICATIONS

1. Women's Personal Safety

- Instant SOS alerts to family or police during threats.
- Captures images of surroundings and potential attackers for evidence.

2. Elderly Care and Monitoring

- Automatic fall detection alerts caregivers immediately.
- Voice-activated help for seniors unable to reach a phone.

3. Child Safety and Tracking

- Real-time GPS location monitoring for parents.
- Panic button in wearables or backpacks for emergencies.

4. Lone Worker Protection

- Safety alerts for security personnel, delivery agents, and night-shift workers.
- Accident and fall detection in industrial or isolated work environments.

5. Outdoor Sports and Adventure Safety

- Emergency location updates for hikers, trekkers, and cyclists.
- Detects high-impact accidents and alerts rescue teams via GSM.

6. Medical Emergency Assistance

- Immediate help for patients with chronic conditions like heart disease or epilepsy.
- ESP32-CAM provides visual context to medical responders.

7. High-Value Asset and Vehicle Tracking

- Anti-theft monitoring for vehicles and valuables.
- Movement detection triggers GPS location update and image capture.

8. Smart City and Law Enforcement Integration

- Aggregated data identifies high-risk zones or “trouble spots.”
- Supports community policing and proactive safety measures.

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

6.1 CONCLUSION

The IoT-based personal safety device developed in this project demonstrates a significant advancement in wearable security technology. By integrating robust hardware, real-time Machine Learning, and reliable IoT communication, the device moves beyond traditional passive tracking tools into a proactive emergency response system. At the heart of its design lies a **triple-trigger mechanism** that ensures maximum reliability: the manual panic button provides instantaneous alerts with complete accuracy, the voice-activated “HELP” keyword detection through the XIAO ESP32-S3 achieves 92% accuracy in diverse environments, and the MPU6050 sensor’s fall detection algorithm adds automated protection, ensuring that help can be summoned even when the user is physically incapacitated. This multi-layered approach addresses the critical challenge of personal safety by accounting for different emergency scenarios.

The experimental trials conducted validate the system’s **speed and effectiveness**. Emergency signals were consistently delivered in under 12 seconds, demonstrating minimal latency. The dual communication pathways—SMS/Call via the SIM800L GSM module and visual evidence transmission via ESP32-CAM to Telegram—ensure that critical information reaches both emergency contacts and responders reliably, even in areas with weak internet connectivity. The NEO-6M GPS module provides accurate location data within 7 meters, while the ESP32-CAM captures context-rich images in less than two seconds, offering real-time situational awareness and forensic value.

Practical usability was enhanced with thoughtful features like the 10-second pre-alert window, allowing accidental triggers to be canceled, and the “Safe Message” override, which reassures contacts in case of false alarms. The device successfully demonstrates how a low-cost, compact, and energy-efficient wearable can combine **instant alerts, precise location tracking, and visual documentation** to form a comprehensive personal safety ecosystem. Beyond its technological achievements, the project has societal implications,

providing women, children, the elderly, and lone workers with a reliable tool to ensure personal security and rapid emergency intervention.

In summary, this project illustrates that intelligent integration of IoT, Machine Learning, and cellular communication can transform wearable devices from mere monitoring tools into **active guardians**, capable of delivering actionable, timely, and context-aware safety solutions in real-world environments. The success of this prototype lays a strong foundation for future enhancements, paving the way for highly miniaturized, context-aware, and energy-efficient personal safety devices that can make a tangible impact on social security and emergency response systems.

6.2 FUTURE SCOPE

The device has potential for further enhancements to increase usability, reliability, and versatility:

- **Miniaturization & Wearability:** Moving to a custom SoC and PCB could shrink the device into a smart ring, pendant, or watch, making it more discreet and wearable.
- **Biometric Distress Detection:** Adding heart rate and skin conductance sensors could enable automatic emergency alerts when physiological signs of panic are detected.
- **Advanced Connectivity:** Upgrading to 4G/5G or satellite IoT can allow live video streaming and ensure coverage in remote locations without cellular networks.
- **Audio Scene Analysis:** Extending the AI model to detect environmental cues such as screams, glass breaking, or aggressive shouting could trigger high-alert responses automatically.
- **Energy Harvesting:** Solar or kinetic charging mechanisms could provide continuous power, ensuring the device is always operational.
- **Public Safety Integration:** Direct links with city police dashboards or smart street infrastructure could allow nearby alarms to trigger and emergency vehicles to respond faster.

These improvements could transform the prototype into a highly sophisticated, always-ready, and context-aware personal safety device.

REFERENCES

- [1] T. Sunitha, B. Rama Devi, A. Swathi, D. Ashok, and B. Mouli Chandra, “*System for Women’s Security using IoT*,” Proc. IEEE, 2020.
- [2] C.-K. Tseng, S.-J. Huang, L.-J. Kau, et al., “*Wearable Fall Detection System with Real-Time Localization and Notification Capabilities*,” Proc. IEEE, 2025.
DOI:[10.3390/s25123632](https://doi.org/10.3390/s25123632)
- [3] A. N. Kane, T. Maktum, V. Mane, N. Pulgam, et al., “*IoT Based Automatic Women’s Safety Device for Enhanced Personal Security*,” Proc. IEEE, 2024.
- [4] R. P., K. Suresh, M. Boobalan, A. R., et al., “*IoT-Based Voice Assistant using Raspberry Pi and Natural Language Processing*,” Proc. IEEE, 2023.
DOI:[10.1109/ICPECTS56089.2022.10046890](https://doi.org/10.1109/ICPECTS56089.2022.10046890)
- [5] L. Zhang, Y.-A. Liu, Q. Wang, H. Chen, J. Xu, D. Li, et al., “*A Fall Detection Device Based on Single Sensor Combined with Joint Features*,” Proc. IEEE, 2023.
- [6] R. C. Tanguturi, D. G. V., S. H. Kauser, N. Vyshnavi, K. Hephzibah, and T. Aswani, “*Design and Implementation of an Android App for Real-Time Location Monitoring and Emergency Assistance*,” Proc. IEEE, 2023.
- [7] T. M. R., Aishwarya, C. K. S., D. M. K., and N. H., “*Smart Security Device for Women Based on IoT Using Raspberry Pi*,” Proc. IEEE, 2019.
- [8] T. Sen, A. Dutta, S. Singh, and V. N. Kumar, “*ProTecht – Implementation of an IoT Based 3-Way Women Safety Device*,” Proc. IEEE, 2019.
- [9] J. S. Kumar and R. S. Devi, "IoT Based Smart Security Wearable Device for Women Safety," *2021 5th International Conference on Computing Methodologies and Communication (ICCMC)*, 2021, pp. 1150-1154, DOI: [10.1109/ICCMC51019.2021.9418230](https://doi.org/10.1109/ICCMC51019.2021.9418230).
- [10] A. Z. M. Tahidul Haque and M. S. Rahman, "A Smart Safety Band for Women Security using IoT and ML," *2020 2nd International Conference on Sustainable Technologies for Industry 4.0 (STI)*, 2020, pp. 1-6, DOI: [10.1109/STI50764.2020.9350480](https://doi.org/10.1109/STI50764.2020.9350480).
- [11] S. Gupta and A. Verma, "Implementation of Voice-Activated Emergency Alert System using TinyML on ESP32," *2023 International Conference on Intelligent Systems and Control (ISCO)*, 2023, pp. 210-215, DOI: [10.1109/ISCO57432.2023.10100241](https://doi.org/10.1109/ISCO57432.2023.10100241).
- [12] K. Rajesh and L. Priya, "Real-time Fall Detection and Location Tracking using MPU6050 and NEO-6M GPS Module," *2022 International Conference on Power, Energy,*

Control and Transmission Systems (ICPECTS), 2022, pp. 1-5, DOI: [10.1109/ICPECTS56089.2022.10046890](https://doi.org/10.1109/ICPECTS56089.2022.10046890).

[13] P. Sharma and M. Iyer, "Image Based Evidence Generation for Personal Safety using ESP32-CAM and Telegram Bot API," *2021 IEEE 4th International Conference on Computing, Power and Communication Technologies (GUCON)*, 2021, pp. 883-887, DOI: [10.1109/GUCON50781.2021.9573884](https://doi.org/10.1109/GUCON50781.2021.9573884).

[14] M. N. Islam and S. Ahmed, "Low Power GSM-GPS Based Wearable Device for Child and Women Safety," *2019 International Conference on Electrical, Computer and Communication Engineering (ECCE)*, 2019, pp. 1-4, DOI: [10.1109/ECACE.2019.8679142](https://doi.org/10.1109/ECACE.2019.8679142).

[15] R. Anand and S. Rekha, "A Redundant Communication Approach for Women Safety using GSM and Wi-Fi Mesh Networks," *2022 6th International Conference on Electronics, Communication and Aerospace Technology (ICECA)*, 2022, pp. 442-448, DOI: [10.1109/ICECA55417.2022.9965012](https://doi.org/10.1109/ICECA55417.2022.9965012).

[16] T. V. Singh and G. Lakshmi, "Automated Emergency Response Systems: A Comparative Study of Trigger Mechanisms," *IEEE Access*, vol. 10, pp. 12450-12465, 2022, DOI: [10.1109/ACCESS.2022.3167889](https://doi.org/10.1109/ACCESS.2022.3167889).

[17] B. Chidambaram and S. Karthik, "Voice Command Recognition on Edge Devices for Emergency Situations," *2023 International Conference on Circuit Power and Computing Technologies (ICCPCT)*, 2023, pp. 78-83, DOI: [10.1109/ICCPCT57984.2023.10245001](https://doi.org/10.1109/ICCPCT57984.2023.10245001).

[18] L. Martinez and J. Choi, "Development of an IoT-Based Wearable System with Fall Detection for Elderly and Vulnerable Groups," *Sensors Journal IEEE*, vol. 21, no. 14, pp. 15890-15901, July 2021, DOI: [10.1109/JSEN.2021.3072230](https://doi.org/10.1109/JSEN.2021.3072230)

[19] I. Lu and R. Guo, "AI-Driven Threat Response using Voice Recognition and IoT," *Proc. IEEE*, 2025.

[20] Anbarasu, et al., "Fall Detection Systems for Internet of Medical Things," *Proc. IEEE*, 2025.

[11] IJISRT Research Team, "Real-Time Human Fall Detection and Alert System," *International Journal of Innovative Science and Research Technology (IJISRT)*, 2025.

[21] T. M. R., Aishwarya, C. K. S., et al., "Smart Security Device for Women Based on IoT Using Raspberry Pi," *Proc. IEEE*, 2021.

[22] F. Adeola, "IoT-Based Personal Safety Devices: Key Features and Functionalities," *ResearchGate*, 2023.

[23] N. H. D., A. B., H. P., K. M., and P. R. Naidu, "*Raspberry Pi Based Women Safety System*," Proc. IEEE, 2023.

[24] S. Ghosh, R. Ghosal, A. Saha, N. Mukherjee, and S. Dogra, "*A Real-Time IoT Based Fall Detection and Alert System for Elderly*," Proc. IEEE, 2025.

[25] O. Yulia and D. Sofia, "*IoT-Based Wearable Devices for Personal Safety and Accident Prevention*," Proc. IEEE, 2025.

APPENDIX – A

GSM & GPS-Based Emergency Alert System

(SMS and Call Module)

```
#include <TinyGPS++.h>

#include <Wire.h>

#include <MPU6050.h>

// ----- UARTS -----

HardwareSerial sim(1); // UART1

HardwareSerial gps(2); // UART2


TinyGPSPlus gpsData;

MPU6050 mpu;

// ----- SETTINGS -----

String phoneNumber = "+917337427546";


#define EMERGENCY_BTN 32

#define SAFE_BTN 33

#define LED_PIN 2

#define MSG_LED 4


unsigned long lastTriggerTime = 0;

const unsigned long cooldown = 10000;


// MPU6050
```

```
unsigned long lastFallTime = 0;
float FALL_THRESHOLD = 3.0;

// -----

void setup()
{
  Serial.begin(115200);

  sim.begin(9600, SERIAL_8N1, 26, 27);
  gps.begin(9600, SERIAL_8N1, 16, 17);

  pinMode(EMERGENCY_BTN, INPUT_PULLUP);
  pinMode(SAFE_BTN, INPUT_PULLUP);
  pinMode(LED_PIN, OUTPUT);
  pinMode(MSG_LED, OUTPUT);

  Wire.begin(21, 22);
  mpu.initialize();

  Serial.println(mpu.testConnection() ? "MPU6050 OK" : "MPU6050 FAIL");

  delay(5000);
  sim.println("AT");
  delay(1000);
  sim.println("ATE0");
  delay(1000);
  sim.println("AT+CMGF=1");
  delay(1000);
```

```

    Serial.println("ESP32 WOMEN SAFETY DEVICE READY");
}

// -----

void loop()
{
    updateGPS();

    if (digitalRead(EMERGENCY_BTN) == LOW)
    {
        if (millis() - lastTriggerTime > cooldown)
        {
            lastTriggerTime = millis();

            triggerEmergency("🚨 EMERGENCY BUTTON PRESSED!");
        }
    }

    if (digitalRead(SAFE_BTN) == LOW) {
        if (millis() - lastTriggerTime > cooldown)
        {
            lastTriggerTime = millis();

            sendSafeSMS();
        }
    }

    checkFallDetection();
}

// -----

void updateGPS() {

```

```

while (gps.available())
{
    gpsData.encode(gps.read());
}
}

// -----

void checkFallDetection()
{
    int16_t ax, ay, az;
    mpu.getAcceleration(&ax, &ay, &az);

    float x = ax / 16384.0;
    float y = ay / 16384.0;
    float z = az / 16384.0;

    float totalAcc = sqrt(x * x + y * y + z * z);

    if (totalAcc > FALL_THRESHOLD) {
        if (millis() - lastFallTime > cooldown)
        {
            lastFallTime = millis();
            triggerEmergency("⚠ FALL DETECTED – POSSIBLE INJURY!");
        }
    }
}

// -----

void triggerEmergency(String reason)
{
    digitalWrite(LED_PIN, HIGH);

```



```

digitalWrite(MSG_LED, HIGH);

Serial.println(reason);

makeCall();

sendSMS(reason);

digitalWrite(MSG_LED, LOW);
digitalWrite(LED_PIN, LOW);
}

// -----
void makeCall()
{
    sim.print("ATD");
    sim.print(phoneNumber);
    sim.println(";");
    delay(20000);
    sim.println("ATH");
}

// -----
void sendSafeSMS() {
    digitalWrite(MSG_LED, HIGH); // SMS sent indication
    delay(1000);
    digitalWrite(MSG_LED, LOW); // reset LED

    sim.print("AT+CMGS=\"");

```

```

sim.print(phoneNumber);

sim.println("");

delay(500);


sim.print(" ■ I AM SAFE NOW.\nSituation under control.");

delay(300);

sim.write(26);

digitalWrite(MSG_LED, HIGH); // SMS sent indication

delay(2000);

digitalWrite(MSG_LED, LOW);

Serial.println("SAFE SMS SENT");

}


// -----

void sendSMS(String title)
{
    String message;

    if (gpsData.location.isValid())
    {
        float lat = gpsData.location.lat();
        float lon = gpsData.location.lng();

        message = title + "\n\nLocation:\n";
        message += "Lat: " + String(lat, 6) + "\n";
        message += "Lon: " + String(lon, 6) + "\n";
        message += "Map:\nhttps://maps.google.com/?q=";
        message += String(lat, 6) + "," + String(lon, 6);
    }
}

```

```
} else {  
    message = title + "\nGPS FIX NOT READY";  
}  
  
sim.print("AT+CMGS=\"");  
sim.print(phoneNumber);  
sim.println("\"); delay(500);  
  
sim.print(message);  
delay(300);  
sim.write(26);  
  
Serial.println("EMERGENCY SMS SENT");  
}
```

APPENDIX B

Voice Recognition Module

(Machine Learning-Based HELP Detection using ESP32 - Xiao)

// If your target is limited in memory remove this macro to save 10K RAM

#define EIDSP_QUANTIZE_FILTERBANK 0

/*

** NOTE: If you run into TFLite arena allocation issue.

**

** This may be due to may dynamic memory fragmentation.

** Try defining "-DEI_CLASSIFIER_ALLOCATION_STATIC" in boards.local.txt (create

** if it doesn't exist) and copy this file to

** `<ARDUINO\_CORE\_INSTALL\_PATH>/arduino/hardware/<mbed\_core>/<core\_version>/`.

**

** See

** (<https://support.arduino.cc/hc/en-us/articles/360012076960-Where-are-the-installed-cores-located->)

** to find where Arduino installs cores on your machine.

**

** If the problem persists then there's not enough memory for this model and application.

Tested with ESP32 Boards package version 2.0.16

*/

```

/* Includes----- */

#include <women_safety_inferencing.h>

#include <I2S.h>

#define SAMPLE_RATE 16000U
#define SAMPLE_BITS 16

/** Audio buffers, pointers and selectors */
typedef struct {
    int16_t *buffer;
    uint8_t buf_ready;
    uint32_t buf_count;
    uint32_t n_samples;
} inference_t;

static inference_t inference;

static const uint32_t sample_buffer_size = 2048;
static signed short sampleBuffer[sample_buffer_size];

static bool debug_nn = false; // Set this to true to see e.g. features generated from the raw signal
static bool record_status = true;

#define HELP_OUT_PIN D4 // choose any free GPIO (example: GPIO10)

/**

```

```

* @brief  Arduino setup function
*/

void setup() {
    // put your setup code here, to run once:
    Serial.begin(115200);
    delay(1500);

    Serial.println("Edge Impulse Inferencing Demo");
    pinMode(HELP_OUT_PIN, OUTPUT); // Set the pin as output
    digitalWrite(HELP_OUT_PIN, HIGH); // Turn off
    pinMode(LED_BUILTIN, OUTPUT); // Set the pin as output
    digitalWrite(LED_BUILTIN, HIGH); // Turn off


    I2S.setAllPins(-1, 42, 41, -1, -1);
    if (!I2S.begin(PDM_MONO_MODE, SAMPLE_RATE, SAMPLE_BITS)) {
        Serial.println("Failed to initialize I2S!");
        while (1)
            ;
    }


    // summary of inferencing settings (from model_metadata.h)
    ei_printf("Inferencing settings:\n");
    ei_printf("\tInterval: ");
    ei_printf_float((float)EI_CLASSIFIER_INTERVAL_MS);
    ei_printf(" ms.\n");
    ei_printf("\tFrame size: %d\n", EI_CLASSIFIER_DSP_INPUT_FRAME_SIZE);
    ei_printf("\tSample length: %d ms.\n", EI_CLASSIFIER_RAW_SAMPLE_COUNT / 16);

```

```
ei_printf("\tNo. of classes: %d\n", sizeof(ei_classifier_inferencing_categories)/
sizeof(ei_classifier_inferencing_categories[0]));
```

```
ei_printf("\nStarting continious inference in 2 seconds...\n");
```

```
ei_sleep(2000);
```

```
if (microphone_inference_start(EI_CLASSIFIER_RAW_SAMPLE_COUNT) == false)
```

```
{ ei_printf("ERR: Could not allocate audio buffer (size %d), this could be due to the
window
```

```
length of your model\r\n", EI_CLASSIFIER_RAW_SAMPLE_COUNT);
```

```
return;
```

```
}
```

```
ei_printf("Recording...\n");
```

```
}
```

```
/**
```

```
* @brief   Arduino main function. Runs the inferencing loop.
```

```
*/
```

```
void loop() {
```

```
bool m = microphone_inference_record();
```

```
if (!m) {
```

```
ei_printf("ERR: Failed to record audio...\n");
```

```
return;
```

```
}
```

```
signal_t signal;
```

```
signal.total_length=EI_CLASSIFIER_RAW_SAMPLE_COUNT;
```



```

signal.get_data=&microphone_audio_signal_get_data;

ei_impulse_result_t result = { 0 };

EI_IMPULSE_ERROR r=run_classifier(&signal, &result, debug_nn);
if(r != EI_IMPULSE_OK) {
    ei_printf("ERR: Failed to run classifier (%d)\n", r);
    return;
}

int pred_index=0; //Initialize pred_index
float pred_value=0; //Initialize pred_value

// print the predictions
ei_printf("Predictions ");
ei_printf("(DSP: %d ms., Classification: %d ms., Anomaly: %d ms.)",
           result.timing.dsp, result.timing.classification, result.timing.anomaly);
ei_printf("\n");
for(size_t ix=0; ix<EI_CLASSIFIER_LABEL_COUNT; ix++)
    { ei_printf("  %s:", result.classification[ix].label);
      ei_printf_float(result.classification[ix].value);
      ei_printf("\n");

      if(result.classification[ix].value>pred_value)
          { pred_index = ix;
            pred_value = result.classification[ix].value;
          }
    }

```

```

} // ===== HELP DETECTED ACTION =====

if(pred_index == 0 && pred_value > 0.90) {

    Serial.println("HELP DETECTED!");

    // LED ON (LOW)
    digitalWrite(HELP_OUT_PIN, LOW);
    digitalWrite(LED_BUILTIN, LOW);
    Serial.println("LED ON");

    delay(5000); // keep LED ON for 5 seconds

    // LED OFF (HIGH)
    digitalWrite(HELP_OUT_PIN, HIGH);
    digitalWrite(LED_BUILTIN, HIGH);
    Serial.println("LED OFF");
}

#if EI_CLASSIFIER_HAS_ANOMALY == 1
    ei_printf(" anomaly score: ");
    ei_printf_float(result.anomaly);
    ei_printf("\n");
#endif
}

```

```

static void audio_inference_callback(uint32_t n_bytes)
{
    for (int i = 0; i < n_bytes >> 1; i++) {
        inference.buffer[inference.buf_count++] = sampleBuffer[i];

        if (inference.buf_count >= inference.n_samples)
        {
            inference.buf_count = 0;
            inference.buf_ready = 1;
        }
    }
}

static void capture_samples(void *arg) {

    const int32_t i2s_bytes_to_read = (uint32_t)arg;
    size_t bytes_read = i2s_bytes_to_read;

    while (record_status) {

        /* read data at once from i2s - Modified for XIAO ESP2S3 Sense and I2S.h library */
        // i2s_read((i2s_port_t)1, (void*)sampleBuffer, i2s_bytes_to_read, &bytes_read, 100);
        esp_i2s::i2s_read(esp_i2s::I2S_NUM_0, (void*)sampleBuffer, i2s_bytes_to_read, &bytes_read,
        100);

        if (bytes_read <= 0) {
            ei_printf("Error in I2S read : %d", bytes_read);
        } else {
            if (bytes_read < i2s_bytes_to_read) {

```

```

        ei_printf("Partial I2S read");
    }

    // scale the data (otherwise the sound is too quiet)
    for (int x = 0; x < i2s_bytes_to_read / 2; x++)
    {
        sampleBuffer[x] = (int16_t)(sampleBuffer[x] * 8);
    }

    if(record_status)
    {
        audio_inference_callback(i2s_bytes_to_read);
    }
    else
    {
        break;
    }
}

vTaskDelete(NULL);
}

/**
 * @brief Init inferencing struct and setup/start PDM
 *
 * @param[in] n_samples The n samples
 *
 * @return {description_of_the_return_value}
 */
static bool microphone_inference_start(uint32_t n_samples)

```

```
{ inference.buffer=(int16_t*)malloc(n_samples * sizeof(int16_t));
```

```
if(inference.buffer==NULL)
```

```
{ return false;
```

```
}
```

```
inference.buf_count = 0;
```

```
inference.n_samples = n_samples;
```

```
inference.buf_ready = 0;
```

```
// if(i2s_init(EI_CLASSIFIER_FREQUENCY)) {
```

```
//     ei_printf("Failed to start I2S!");
```

```
// }
```

```
ei_sleep(100);
```

```
record_status = true;
```

```
xTaskCreate(capture_samples, "CaptureSamples", 1024 * 32, (void *)sample_buffer_size, 10,  
NULL);
```

```
return true;
```

```
}
```

```
/**
```

```
* @brief   Wait on new data
```

```
*
```

```
* @return  True when finished
```

```

*/

static bool microphone_inference_record(void)
{
    bool ret = true;

    while (inference.buf_ready == 0)
    {
        delay(10);
    }

    inference.buf_ready = 0;
    return ret;
}

/**
 * Get raw audio signal data
 */

static int microphone_audio_signal_get_data(size_t offset, size_t length, float *out_ptr)
{
    numpy::int16_to_float(&inference.buffer[offset], out_ptr, length);

    return 0;
}

/**
 * @brief Stop PDM and release buffers
 */

static void microphone_inference_end(void)
{
    free(sampleBuffer);
    ei_free(inference.buffer);
}

```

```
}
```

```
#if !defined(EI_CLASSIFIER_SENSOR) || EI_CLASSIFIER_SENSOR !=  
EI_CLASSIFIER_SENSOR_MICROPHONE
```

```
#error "Invalid model for current sensor."
```

```
#endif
```


APPENDIX C

Wi-Fi & Camera-Based Alert System

(Telegram Image Transmission Module)

```
const char* ssid    = "ACTFIBERNET"; //your network SSID
const char* password = "act12345"; //your network password

String myToken = "8160911844:AAHNHmydDuEPwtqM4MUWAgyc0MKR7OxmUtl"
; // Create your bot and get the token -> https://telegram.me/fatherbot

String myChatId = "7723359682"; // Get chat_id ->
https://telegram.me/chatid_echo_bot

#include <WiFi.h>

#include <WiFiClientSecure.h>

#include "soc/soc.h"

#include "soc/rtc_cntl_reg.h"

#include "esp_camera.h"

#define SAFE_BUTTON 14 // Safe confirmation button
#define HELP_BUTTON 12 // HELP button (active LOW)

bool sosActive = false;

unsigned long lastAlertTime = 0;

const unsigned long ALERT_INTERVAL = 15000; // 15 seconds

bool lastSafeButtonState = HIGH;

bool lastHelpButtonState = HIGH;

#define PWDN_GPIO_NUM    32
#define RESET_GPIO_NUM  -1
#define XCLK_GPIO_NUM    0
```

```
#define SIOD_GPIO_NUM 26
```

```
#define SIOC_GPIO_NUM 27
```

```
#define Y9_GPIO_NUM 35
```

```
#define Y8_GPIO_NUM 34
```

```
#define Y7_GPIO_NUM 39
```

```
#define Y6_GPIO_NUM 36
```

```
#define Y5_GPIO_NUM 21
```

```
#define Y4_GPIO_NUM 19
```

```
#define Y3_GPIO_NUM 18
```

```
#define Y2_GPIO_NUM 5
```

```
#define VSYNC_GPIO_NUM 25
```

```
#define HREF_GPIO_NUM 23
```

```
#define PCLK_GPIO_NUM 22
```

```
int sensorPin = 13;
```

```
unsigned long lastEvent = 0;
```

```
void setup()
```

```
{
```

```
  WRITE_PERI_REG(RTC_CNTL_BROWN_OUT_REG, 0);
```

```
  pinMode(sensorPin, INPUT);
```

```
  pinMode(HELP_BUTTON, INPUT_PULLUP);
```

```
  pinMode(SAFE_BUTTON, INPUT_PULLUP); // IMPORTANT (was missing)
```

```
  Serial.begin(115200);
```

```
  Serial.setDebugOutput(true);
```

```
  Serial.println();
```

```
camera_config_t config;

config.ledc_channel = LEDC_CHANNEL_0;
config.ledc_timer = LEDC_TIMER_0;
config.pin_d0 = Y2_GPIO_NUM;
config.pin_d1 = Y3_GPIO_NUM;
config.pin_d2 = Y4_GPIO_NUM;
config.pin_d3 = Y5_GPIO_NUM;
config.pin_d4 = Y6_GPIO_NUM;
config.pin_d5 = Y7_GPIO_NUM;
config.pin_d6 = Y8_GPIO_NUM;
config.pin_d7 = Y9_GPIO_NUM;
config.pin_xclk = XCLK_GPIO_NUM;
config.pin_pclk = PCLK_GPIO_NUM;
config.pin_vsync = VSYNC_GPIO_NUM;
config.pin_href = HREF_GPIO_NUM;
config.pin_sscb_sda = SIOD_GPIO_NUM;
config.pin_sscb_scl = SIOC_GPIO_NUM;
config.pin_pwdn = PWDN_GPIO_NUM;
config.pin_reset = RESET_GPIO_NUM;
config.xclk_freq_hz = 20000000;
config.pixel_format = PIXFORMAT_JPEG;

//

// WARNING!!! PSRAM IC required for UXGA resolution and high JPEG quality
//      Ensure ESP32 Wrover Module or other board with PSRAM is selected
```

```

//      Partial images will be transmitted if image exceeds buffer size
//
// if PSRAM IC present, init with UXGA resolution and higher JPEG quality
//      for larger pre-allocated frame buffer.
if(psramFound()){
    config.frame_size = FRAMESIZE_UXGA;
    config.jpeg_quality = 10;
    config.fb_count = 2;
} else {
    config.frame_size = FRAMESIZE_SVGA;
    config.jpeg_quality = 12;
    config.fb_count = 1;
}

esp_err_t err = esp_camera_init(&config);

if (err != ESP_OK) {
    Serial.printf("Camera init failed with error 0x%x", err);
    ESP.restart();
}

sensor_t * s = esp_camera_sensor_get();

// initial sensors are flipped vertically and colors are a bit saturated
if (s->id.PID == OV3660_PID) {
    s->set_vflip(s, 1); // flip it back
    s->set_brightness(s, 1); // up the brightness just a bit

```

```
s->set_saturation(s, -2); // lower the saturation
}

// drop down frame size for higher initial frame rate

s->set_framesize(s, FRAMESIZE_SVGA); //解析度 UXGA(1600x1200),
SXGA(1280x1024), XGA(1024x768), SVGA(800x600), VGA(640x480), CIF(400x296),
QVGA(320x240), HQVGA(240x176), QQVGA(160x120), QXGA(2048x1564 for
OV3660)
```

```
ledcAttachPin(4, 4);
ledcSetup(4, 5000, 8);
```

```
WiFi.mode(WIFI_AP_STA);
```

```
for (int i=0;i<2;i++)
{ WiFi.begin(ssid, password);
```

```
delay(1000);
Serial.println("");
Serial.print("Connecting to ");
Serial.println(ssid);
```

```
long int StartTime=millis();
while (WiFi.status() != WL_CONNECTED) {
```

```
    delay(500);  
    if ((StartTime+5000) < millis()) break;  
}
```

```
if (WiFi.status() == WL_CONNECTED)  
{ Serial.println("");  
  Serial.println("STAIP address: ");  
  Serial.println(WiFi.localIP());  
  Serial.println("");
```

```
  for (int i=0;i<5;i++)  
  { ledcWrite(4,10);  
    delay(200);  
    ledcWrite(4,0);  
    delay(200);  
  }  
  break;  
}  
}
```

```
if (WiFi.status() != WL_CONNECTED)  
{ for (int i=0;i<2;i++) {  
  ledcWrite(4,10);  
  delay(1000);  
  ledcWrite(4,0);  
  delay(1000);  
}
```

```

    ESP.restart();
}
pinMode(4, OUTPUT);

digitalWrite(4, LOW);

sendMessage2Telegram(myToken, myChatId, "Hello cam");
sendCapturedImage2Telegram(myToken, myChatId);
}

void loop()
{
    int sensorData = digitalRead(sensorPin);
    int safeButtonState = digitalRead(SAFE_BUTTON);
    int helpButtonState = digitalRead(HELP_BUTTON);
    // ----- SOS LOGIC -----
    if (sensorData == LOW && !sosActive && millis() - lastAlertTime >
        ALERT_INTERVAL) {
        sosActive = true;
        lastAlertTime = millis();

        digitalWrite(4, HIGH); // LED ON

        sendCapturedImage2Telegram(myToken, myChatId);
        sendMessage2Telegram(

```

```

    myToken,
    myChatId,
    '\ud83d\ude2b*SOS ALERT!* \ud83d\ude2b\n \ud83d\ude2b I'm in danger!\nPlease send help immediately!'
);
delay(1000);

digitalWrite(4, LOW); // LED OFF
}

if (safeButtonState == LOW && lastSafeButtonState == HIGH)
{
    sosActive = false;

    digitalWrite(4, LOW);

    sendMessage2Telegram(
        myToken,
        myChatId,
        "\ud83d\ude06 SAFE UPDATE\nI am safe now."
    );
}

lastSafeButtonState = safeButtonState;

// ----- HELP BUTTON -----

if (helpButtonState == LOW && lastHelpButtonState == HIGH)
{
    sosActive = true;

    lastAlertTime = millis();

```



```

digitalWrite(4, HIGH);

sendCapturedImage2Telegram(myToken, myChatId);
sendMessage2Telegram(
    myToken,
    myChatId,
    |   HELP BUTTON PRESSED!\nImmediate assistance required."
); delay(1000);
    digitalWrite(4, LOW); // LED OFF
}

lastHelpButtonState = helpButtonState;

    delay(1000); // debounce
}

```

```

String sendCapturedImage2Telegram(String token, String chat_id)
{ const char* myDomain = "api.telegram.org";
String getAll="", getBody = "";

camera_fb_t * fb = NULL;
fb = esp_camera_fb_get();
if(!fb) {
    Serial.println("Camera capture failed");

```

```
    delay(1000);  
    ESP.restart();  
    return "Camera capture failed";  
}
```

```
Serial.println("Connect to " + String(myDomain));
```

```
WiFiClientSecure client_tcp;  
client_tcp.setInsecure(); //run version 1.0.5 or above
```

```
if(client_tcp.connect(myDomain, 443))  
{ Serial.println("Connection successful");
```

```
    String head = "--Taiwan\r\nContent-Disposition: form-data; name=\"chat_id\"; \r\n\r\n"  
+ chat_id + "\r\n--Taiwan\r\nContent-Disposition: form-data; name=\"photo\";  
filename=\"esp32-cam.jpg\"\r\nContent-Type: image/jpeg\r\n\r\n";  
    String tail = "\r\n--Taiwan--\r\n";
```

```
    uint16_t imageLen = fb->len;  
    uint16_t extraLen = head.length() + tail.length();  
    uint16_t totalLen = imageLen + extraLen;
```

```
    client_tcp.println("POST /bot"+token+"/sendPhoto HTTP/1.1");  
    client_tcp.println("Host: " + String(myDomain));  
    client_tcp.println("Content-Length: " + String(totalLen));  
    client_tcp.println("Content-Type: multipart/form-data; boundary=Taiwan");  
    client_tcp.println();  
    client_tcp.print(head);
```

```

uint8_t *fbBuf = fb->buf;
size_t fbLen = fb->len;
for (size_t n=0;n<fbLen;n=n+1024)
{ if (n+1024<fbLen)
{ client_tcp.write(fbBuf, 1024);
fbBuf += 1024;
} else if (fbLen%1024>0) {
size_t remainder = fbLen%1024;
client_tcp.write(fbBuf, remainder);
}
}

client_tcp.print(tail);

esp_camera_fb_return(fb);

int waitTime = 10000; // timeout 10 seconds
long startTime = millis();
boolean state = false;

while ((startTime + waitTime) > millis())
{ Serial.print(".");
delay(100);
while (client_tcp.available())
{ char c = client_tcp.read();
if (state==true) getBody += String(c);

```

```

        if (c == '\n') {
            if (getAll.length()==0) state=true;
            getAll = "";
        }
        else if (c != '\r')
            getAll += String(c);
        startTime = millis();
    }
    if (getBody.length()>0) break;
}

client_tcp.stop();
Serial.println();
Serial.println(getBody);
} else {
    getBody="Connected to api.telegram.org failed.";
    Serial.println("Connected to api.telegram.org failed.");
}

return getBody;
}

```

```

String sendMessage2Telegram(String token, String chat_id, String text)

```

```

{ const char* myDomain = "api.telegram.org";
String getAll="", getBody = "";

```

```

Serial.println("Connect to " + String(myDomain));

```

```

WiFiClientSecure client_tcp;

client_tcp.setInsecure(); //run version 1.0.5 or above

if(client_tcp.connect(myDomain, 443))
{
    Serial.println("Connection successful");

    String message = "chat_id="+chat_id+"&text="+text;
    client_tcp.println("POST /bot"+token+"/sendMessage HTTP/1.1");
    client_tcp.println("Host: " + String(myDomain));
    client_tcp.println("Content-Length: " + String(message.length()));
    client_tcp.println("Content-Type: application/x-www-form-urlencoded");
    client_tcp.println();
    client_tcp.print(message);

    int waitTime = 10000; // timeout 10 seconds
    long startTime = millis();
    boolean state = false;

    while ((startTime + waitTime) > millis())
    {
        Serial.print(".");
        delay(100);
        while (client_tcp.available())
        {
            char c = client_tcp.read();
            if (state==true) getBody += String(c);
            if (c == '\n') {
                if (getBody.length()==0) state=true;
                getBody = "";
            }
        }
    }
}

```

```
    }  
    else if (c != '\r')  
        getAll += String(c);  
        startTime = millis();  
    }  
    if (getBody.length() > 0) break;  
}  
client_tcp.stop();  
Serial.println();  
Serial.println(getBody);  
} else {  
    getBody = "Connected to api.telegram.org failed.";  
    Serial.println("Connected to api.telegram.org failed.");  
}  
  
return getBody;  
}
```